Developer Guide

Plug-In Development
2026 R1

Acumatica
The Cloud ERP

Contents

Copyright 4

Plug-In Development Guide 5

Creating Widgets for Dashboards 6

Widget Creation 6

Use of Widgets 7

To Create a Simple Widget 9

To Create an Inquiry-Based Widget 11

To Load a Widget Synchronously or Asynchronously 13

To Add a Script to a Widget 14

To Add Custom Controls to the Widget Properties Dialog Box 14

Customizing Business Events in Code 17

To Define a Custom Subscriber Type for Business Events 17

Adding a Custom SMS Provider 23

Custom SMS Provider: General Information 23

Custom SMS Provider: To Add a Custom SMS Provider 24

Implementing Plug-Ins for Processing Credit Card Payments 30

Interfaces for Processing Credit Card Payments 30

To Implement a Plug-In for Processing Credit Card Payments 31

Implementing a Connector for an External System 35

Connector Implementation: General Information 35

Connector Implementation: Required Libraries 37

Connector Implementation: Classes for Acumatica ERP Entities 38

Connector Implementation: Classes for External Entities 39

Connector Implementation: Mapping Classes 42

Connector Implementation: Bucket Classes 43

Connector Implementation: DAC for the Connector's Configuration Settings 43

Connector Implementation: API Client 46

Connector Implementation: Processor Classes 48

Connector Implementation: Connector Class 57

Connector Implementation: Connector's Factory Class 60

Connector Implementation: How the Connector Is Detected in the Application 61

Connector Implementation: How the Connection with the External System Is Established 62

Connector Implementation: How Data Is Prepared for Synchronization 63

Connector Implementation: How the Prepared Data Is Synchronized 65

Implementing Real-Time Synchronization for a Connector..... 70

Real-Time Synchronization for a Connector: General Information..... 70

Real-Time Synchronization for a Connector: How Webhooks and Push Notifications Are Used..... 72

Copyright

© 2026 Acumatica, Inc.

ALL RIGHTS RESERVED.

No part of this document may be reproduced, copied, or transmitted without the express prior consent of Acumatica, Inc.

3075 112th Avenue NE, Suite 200, Bellevue, WA 98004, USA

Restricted Rights

The product is provided with restricted rights. Use, duplication, or disclosure by the United States Government is subject to restrictions as set forth in the applicable License and Services Agreement and in subparagraph (c)(1)(ii) of the Rights in Technical Data and Computer Software clause at DFARS 252.227-7013 or subparagraphs (c)(1) and (c)(2) of the Commercial Computer Software-Restricted Rights at 48 CFR 52.227-19, as applicable.

Disclaimer

Acumatica, Inc. makes no representations or warranties with respect to the contents or use of this document, and specifically disclaims any express or implied warranties of merchantability or fitness for any particular purpose. Further, Acumatica, Inc. reserves the right to revise this document and make changes in its content at any time, without obligation to notify any person or entity of such revisions or changes.

Trademarks

Acumatica is a registered trademark of Acumatica, Inc. HubSpot is a registered trademark of HubSpot, Inc. Microsoft Exchange and Microsoft Exchange Server are registered trademarks of Microsoft Corporation. All other product names and services herein are trademarks or service marks of their respective companies.

Software Version: 2026 R1

Last Updated: 03/15/2026

Plug-In Development Guide

In this guide, you can find information about how to expand Acumatica ERP functionality by developing custom plug-ins.

Creating Widgets for Dashboards

In Acumatica ERP or an Acumatica Framework-based application, a *dashboard* is a collection of widgets that are displayed on a single page to give you information at a glance. A *widget* is a small component that delivers a particular type of information. Acumatica ERP and any Acumatica Framework-based application support a number of predefined types of widgets that you can add to dashboards. For more information on the supported widgets and their types, see *Configuring Widgets*.

If none of the predefined widget types suits your task, you can create custom widgets, as the topics in this chapter describe, and use these widgets in the dashboards of Acumatica ERP or an Acumatica Framework-based application.

Widget Creation

When you create a custom widget to be used in Acumatica ERP or an Acumatica Framework-based application, you must implement at least the three classes described in the sections below. (For details on the implementation of the classes, see *To Create a Simple Widget* and *To Create an Inquiry-Based Widget*.)

Data Access Class with the Widget Parameters

The data access class (DAC) with the widget parameters is a general DAC that extends the `PXBqlTable` class and implements the `IBqlTable` interface.

You can use any DAC attributes with this DAC. However, only the fields with `PXDB` attributes, such as `PXDBString` and `PXDBInt`, are stored in the database. The values of these fields are stored in the database automatically; you do not need to create a database table for them.

The fields of the DAC are displayed as controls in the **Widget Properties** dialog box when a user adds a new widget instance to a dashboard or modifies the parameters of an existing widget instance. (The way the controls are displayed depends on *PXFieldState* of the corresponding fields.) For information on adding comprehensive controls to the dialog box, see *To Add Custom Controls to the Widget Properties Dialog Box*.

Graph for the Widget

The graph for the widget is used to manage the widget parameters and read data for the widget. The graph must be inherited from the `PX.Dashboards.WizardMaint` class. For the widget graph, you can manage widget parameters by defining new events and redefining the events that are available in the base classes, such as the `RowSelected` and `FieldSelecting` events. (You work with the events of the widget graph in the same way as you work with the events of a general graph.)

Class of the Widget

The widget class is used by the system to work with the widget instances. The widget class must implement the `PX.Web.UI.IDashboardWidget` interface. The system treats as widgets the classes that implement this interface and are available in a library in the `Bin` folder of an instance of Acumatica ERP or Acumatica Framework-based application. The caption and description of the widget from these classes are displayed in the **Add Widget** dialog box automatically when a user adds a new widget to a dashboard. The methods of this class are used by the system to manage the parameters of a widget instance and display the widget on a dashboard page.

Related Links

- *To Create a Simple Widget*
- *To Create an Inquiry-Based Widget*

Use of Widgets

In this topic, you will learn how a custom widget is used in Acumatica ERP or an Acumatica Framework-based application. For more information on how to work with widgets on a dashboard page, see *Configuring Widgets*.

How the Widget Is Detected in an Application

Once the library that contains the class of the widget is placed in the `Bin` folder of an instance of Acumatica ERP or an Acumatica Framework-based application, the **Add Widget** dialog box, which you open to add a new widget to a dashboard page, contains the caption and description of the new widget. The system takes the caption and description of the widget from the implementation of the `Caption` and `Description` properties of the `IDashboardWidget` interface.

The following diagram illustrates the relations of the class of the widget and the **Add Widget** dialog box. In the diagram, the items that you add for the custom widget are shown in rectangles with a red border.

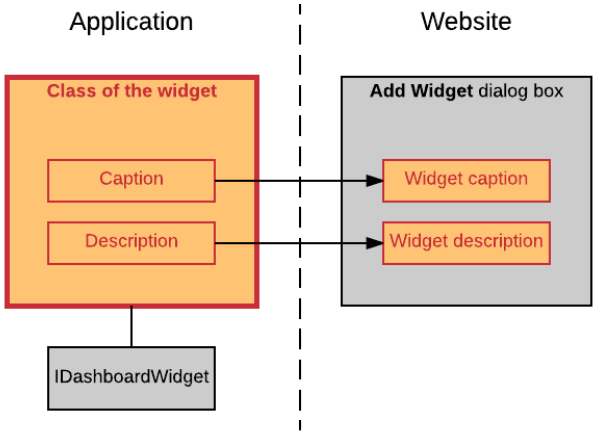


Figure: Widget detection

How a Widget Instance Is Configured

When you select the widget in the **Add Widget** dialog box and click **Next**, the controls for the parameters that should be configured for a new widget instance—that is, the predefined **Caption** box and the controls for the parameters, which are defined in the custom DAC—are displayed in the **Widget Properties** dialog box. These controls are maintained by using the graph for the widget, to which the reference is provided by the class of the widget. The graph uses the `Widget` DAC and the custom DAC with the widget parameters to save the parameters of a widget instance in the `Widget` table of the application database.

To display custom controls, such as buttons and pop-up panels, in the **Widget Properties** dialog box, the system uses the implementations of the `RenderSettings()` and `RenderSettingsComplete()` methods of the `IDashboardWidget` interface from the class of the widget. For more information on adding custom controls to the dialog box, see *To Add Custom Controls to the Widget Properties Dialog Box*.

The following diagram illustrates the interaction of the components of the widget with the **Widget Properties** dialog box. In the diagram, the items that you add for the custom widget are shown in rectangles with a red border.

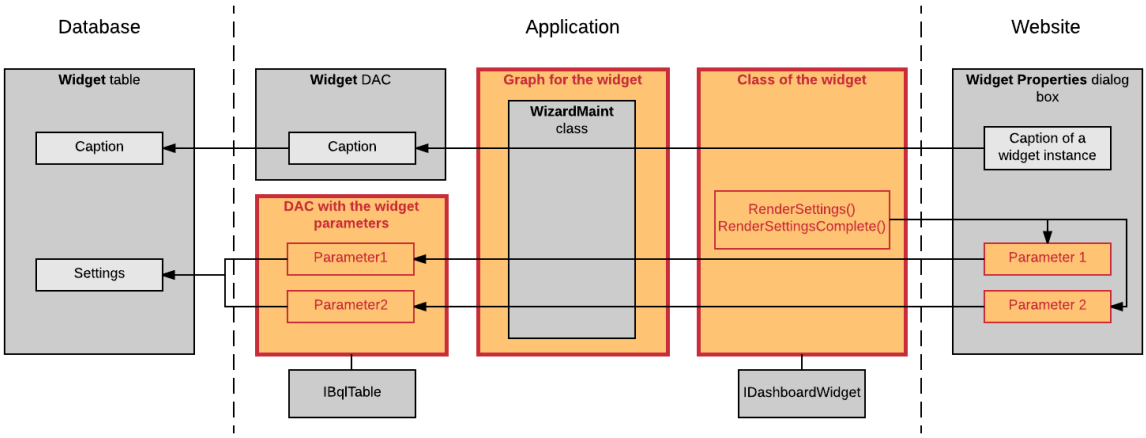


Figure: Configuration of a widget instance

How a Widget Instance Is Displayed on a Dashboard Page

After a widget instance is configured and saved in the **Widget Properties** dialog box, the widget instance is displayed on the dashboard page. To display the caption of the widget instance on the dashboad page, the widget graph retrieves the caption from the `Widget` table of the database by using the `Widget` DAC. To display the widget contents, the system uses the implementations of the `Render()` and `RenderComplete()` methods of the `IDashboardWidget` interface from the class of the widget. These implementations use the parameters of the widget, which the widget graph retrieves from the `Widget` table of the application database by using the DAC with the widget parameters.

You can specify how the widget should be loaded to a dashboard page and implement custom scripts to manage the way the widget is displayed on a dashboard page. For more information, see *To Load a Widget Synchronously or Asynchronously* and *To Add a Script to a Widget*.

The following diagram illustrates the interaction of the components of the widget with a dashboard page. In the diagram, the items that you add for the custom widget are shown in rectangles with a red border.

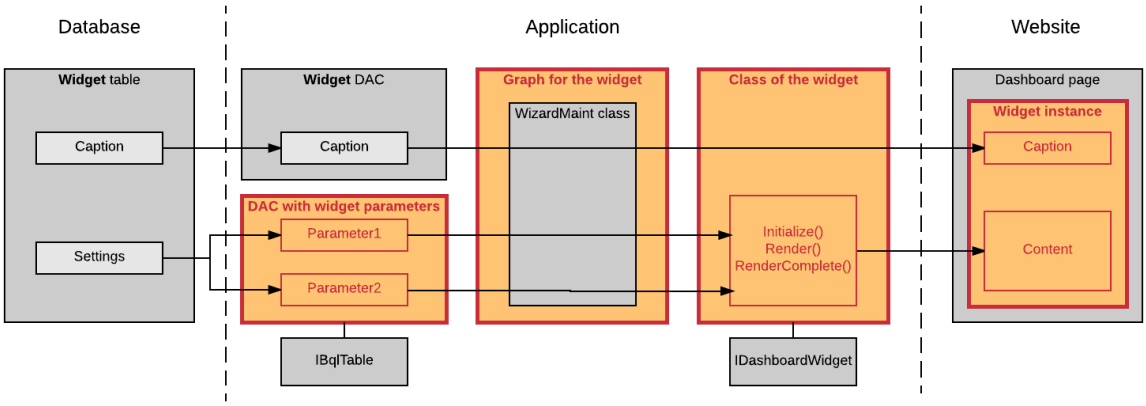


Figure: Widget instance on a dashboard page

Related Links

- *Designing Dashboard Contents*
- *Administering Dashboards*
- *Configuring Widgets*

To Create a Simple Widget

A simple widget displays information from a single data source and does not require calculations or comprehensive data retrieval. Multiple simple widgets, such as wiki widgets or embedded page widgets, are available in Acumatica ERP by default.

To create a simple widget, you need to perform the basic steps that are described in this topic.

To Create a Simple Widget

1. In the project of your Acumatica ERP extension library or your Acumatica Framework-based application, create a data access class (DAC), which stores the parameters of the widget. The DAC must extend the `PXBqlTable` class and implement the `IBqlTable` interface; you can use any DAC attributes with this DAC.

   The following code fragment shows an example of a DAC for a simple widget, which uses one parameter.

```
using PX.Data;

[PXHidden]
public class YFSettings : PXBqlTable, IBqlTable
{
    #region PagePath
    [PXDBString]
    [PXDefault]
    [PXUIField(DisplayName = "Help Article")]
    public string PagePath { get; set; }
    public abstract class pagePath : IBqlField { }
    #endregion
}
```

2. In the project, create a graph for working with the parameters of the widget and reading the data for the widget. The graph must be inherited from the `PX.Dashboards.WizardMaint` class.

   The following code fragment shows an example of a graph for a widget.

```
using PX.Dashboards;

public class YFSettingsMaint : WizardMaint<YFSettingsMaint, YFSettings>
{
}
```

3. In the project, create a widget class that implements the `PX.Web.UI.IDashboardWidget` interface. Use the following instructions for implementation:
   - Inherit the widget class from the `PX.Dashboards.Widgets.PXWidgetBase` abstract class. This class implements part of the required functionality of the `IDashboardWidget` interface, such as localization of the caption and the description of the widget (which are displayed in the **Add Widget** dialog box when a user adds a new widget to a dashboard). This class also stores useful properties of the widget, such as `Settings`, `DataGraph`, `Page`, `DataSource`, and `WidgetID`.
   - Store the values of the caption and the description of the widget in a `Messages` class that has the `PXLocalizable` attribute. This approach is required for localization functionality to work properly.
   - Perform initialization of a widget class instance in the `Initialize()` method of the `IDashboardWidget` interface.

- Create the tree of controls of the widget in the `Render()` method of the `IDashboardWidget` interface.
- If you need to check the access rights of a user to the data displayed in the widget, implement the `IsAccessible()` method of the `IDashboardWidget` interface. If the user has access to the data in the widget, the method must return `true`; if the user has insufficient rights to access the data in the widget, the method must return `false`.
- If you want to specify the way the widget is loaded, override the `AsyncLoading()` method of the `PXWidgetBase` abstract class, as described in *To Load a Widget Synchronously or Asynchronously*.

The following code fragment gives an example of a widget class. This class is inherited from the `PXWidgetBase` class. The caption and description of the widget are specified in the `Messages` class, which has `PXLocalizable` attribute. The widget class implements the `Render()` method to create the control of the widget and performs the configuration of the control in the `RenderComplete()` method.

```
using PX.Web.UI;
using PX.Dashboards.Widgets;
using PX.Common;
using System;
using System.Web.UI;
using System.Web.UI.WebControls;

public class YFWidget : PXWidgetBase<YFSettingsMaint, YFSettings>
{
    public YFWidget()
        : base(Messages.YFWidgetCaption, Messages.YFWidgetDescription)
    {
    }

    protected override WebControl Render(PXDataSource ds, int height)
    {
        if (String.IsNullOrEmpty(Settings.PagePath)) return null;

        WebControl frame = _frame = new WebControl(HtmlTextWriterTag.Iframe)
        { CssClass = "iframe" };
        frame.Attributes["frameborder"] = "0";
        frame.Width = frame.Height = Unit.Percentage(100);
        frame.Attributes["src"] = "javascript:void 0";
        return frame;
    }

    public override void RenderComplete()
    {
        if (_frame != null)
        {
            var renderer = JSManager.GetRenderer(this.Page);
            renderer.RegisterStartupScript(this.GetType(),
                this.GenerateControlID(this.WidgetID),
                string.Format("px.elemByID('{0}').src = '{1}';",
                  _frame.ClientID, Settings.PagePath), true);
        }
        base.RenderComplete();
    }

    private WebControl _frame;
}

[PXLocalizable]
public static class Messages
```

```
{
    public const string YFWidgetCaption = "YogiFon Help Page";
    public const string YFWidgetDescription = "Displays a Help page.";
}
```

4. Compile your Acumatica ERP extension library.
5. Run the application and make sure that the new widget appears in the **Add Widget** dialog box. The widget class, which implements the `IDashboardWidget` interface, is detected by the system and automatically added to the list of widgets available for selection in the dialog box.

Related Links

- *To Load a Widget Synchronously or Asynchronously*

To Create an Inquiry-Based Widget

An inquiry widget retrieves data for the widget from an inquiry form, such as a generic inquiry form or a custom inquiry form. Inquiry-based widgets that are available in Acumatica ERP or an Acumatica Framework-based application by default include chart widgets, table widgets, and KPI widgets.

To create an inquiry-based widget, you need to perform the basic steps that are described in the section below. In these steps, you use the predefined classes, which provide the following functionality to simplify the development of an inquiry-based widget:

- Selection of the inquiry form in the widget settings
- Selection of a shared inquiry filter
- Selection of the parameters of the inquiry
- The ability to drill down in the inquiry form when a user clicks on the widget caption
- Verification of the user's access rights to the widget, which is performed based on the user's access rights to the inquiry form

To Create an Inquiry-Based Widget

1. In the project of your Acumatica ERP extension library or your Acumatica Framework-based application, create a data access class (DAC) that implements the `IBqlTable` interface and stores the parameters of the widget. We recommend that you inherit the DAC from the `PX.Dashboards.Widgets.InquiryBasedWidgetSettings` class, which provides the following parameters for the widget:
   - *InquiryScreenID*: Specifies the inquiry form on which the widget is based
   - *FilterID*: Specifies one of the shared filters available for the specified inquiry form

   The following code shows a fragment of the DAC for the predefined inquiry-based data table widget. The DAC is inherited from the `InquiryBasedWidgetSettings` class.

```
using PX.Data;
using PX.Dashboards.Widgets;

[PXHidden]
public class TableSettings : InquiryBasedWidgetSettings, IBqlTable
{
    #region AutoHeight
    [PXDBBool]
    [PXDefault(true)]
    [PXUIField(DisplayName = "Automatically Adjust Height")]
```

```
    public bool AutoHeight { get; set; }
    public abstract class autoHeight : IBqlField { }
    #endregion

    ...
}
```

> You must declare a DAC by extending the `PXBqlTable` class and implementing the `IBqlTable` interface. However, in the code example above, the `InquiryBasedWidgetSettings` class already inherits the `PXBqlTable` class.

2. In the project, create a graph for working with widget parameters and reading data for the widget. Use the following instructions when you implement the graph:
   - Inherit the graph from the `PX.Dashboards.Widgets.InquiryBasedWidgetMaint` abstract class, which is inherited from the `PXWidgetBase` abstract class.
   - Implement the `SettingsRowSelected()` event, which is the `RowSelected` event for the DAC with widget parameters; it contains the current values of the parameters of the widget instance and the list of available fields of the inquiry form. (For information on how to work with the fields of the inquiry form, see *To Use the Parameters and Fields of the Inquiry Form in the Widget*.) The signature of the method is shown below.

     ```
     protected virtual void SettingsRowSelected(PXCache cache,
       TPrimary settings, InqField[] inqFields)
     ```

3. In the project, create a widget class. We recommend that you inherit this class from the `PX.Dashboards.Widgets.InquiryBasedWidget` class.
4. Compile your Acumatica ERP extension library or Acumatica Framework-based application.
5. Run the application, and make sure that the new widget appears in the **Add Widget** dialog box. The widget class, which implements the `IDashboardWidget` interface, is detected by the system and automatically added to the list of widgets available for selection in the dialog box.

To Use the Parameters and Fields of the Inquiry Form in the Widget

You can access the parameters and fields of the inquiry from that is used in the widget by using the `DataScreenBase` class, which is available through the `DataScreen` property in the widget graph and in the widget class. An instance of the `DataScreenBase` class, which is created based on the inquiry form selected by a user in the widget settings, contains the following properties:

- `ViewName`: Specifies the name of the data view from which the data for the widget is taken.
- `View`: Returns the data view from which the data for the widget is taken.
- `ParametersViewName`: Specifies the name of the data view with the parameters of the inquiry.
- `ParametersView`: Returns the data view with the parameters of the inquiry. It can be `null` if the inquiry has no parameters.
- `ScreenID`: Specifies the ID of the inquiry form.
- `DefaultAction`: Specifies the action that is performed when a user double-clicks on the row in the details table of the inquiry form.

To access the fields of the inquiry form in the widget, use the `GetFields()` method of the `DataScreenBase` class. This method returns the `InqField` class, which provides the following properties:

- `Name`: Specifies the internal name of the field
- `DisplayName`: Specifies the name of the field as it is displayed in the UI
- `FieldType`: Specifies the C# type of the field

- `Visible`: Specifies whether the field is visible in the UI
- `Enabled`: Specifies whether the field is enabled in the UI
- `LinkCommand`: Specifies the linked command of the field

To access the parameters of the inquiry form in the widget, use the `GetParameters()` method of the `DataScreenBase` class, which returns the `InqField` class.

To Load a Widget Synchronously or Asynchronously

By default, if a widget class is inherited from the `PX.Dashboards.Widgets.PXWidgetBase` abstract class, the widget is loaded asynchronously after the page has been loaded. You can change this behavior by using the `AsyncLoading()` method of the widget class, as described in the following sections.

To Load a Widget Synchronously

To load a widget synchronously along with the page, override the `AsyncLoading()` method, as the following code shows.

```
using PX.Web.UI;
using PX.Dashboards.Widgets;

public override AsyncLoadMode AsyncLoading
{
    get { return AsyncLoadMode.False; }
}
```

To Load a Widget Asynchronously

To load a widget asynchronously after the page has been loaded, you do not need to perform any actions, because the following implementation of the `AsyncLoading()` method is used by default.

```
using PX.Web.UI;
using PX.Dashboards.Widgets;

public override AsyncLoadMode AsyncLoading
{
    get { return AsyncLoadMode.True; }
}
```

To Load a Widget that Performs a Long-Running Operation

If a widget performs a long-running operation during loading, such as reading data that takes a long time, use the following approach to load this widget:

1. Override the `AsyncLoading()` method, as the following code shows. In this case, for processing the data of the widget, the system starts the long-running operation in a separate thread.

```
using PX.Web.UI;
using PX.Dashboards.Widgets;

public override AsyncLoadMode AsyncLoading
{
    get { return AsyncLoadMode.LongRun; }
```

```
}
```

2. Override the `ProcessData()` method of the widget class so that it implements all the logic for the widget that consumes significant time while loading.
3. Override the `SetProcessResult()` method of the widget class so that it retrieves the result of the processing of the widget data.

If these methods are implemented, the system calls them automatically when it loads the widget on a dashboard page.

To Add a Script to a Widget

You can specify how a widget should be displayed on a dashboard page by using a custom script. For example, you can implement a script that handles the way the widget is resized.

To add a custom script to a widget, you override the `RegisterScriptModules()` method of the `PX.Dashboards.Widgets.PXWidgetBase` abstract class. The following code shows an example of the method implementation for a predefined data table widget.

```
using PS.Web.UI;
using PX.Dashboards.Widgets;

internal const string JSResource =
  "PX.Dashboards.Widgets.Table.px_dashboardGrid.js";

public override void RegisterScriptModules(Page page)
{
    var renderer = JSManager.GetRenderer(page);
    renderer.RegisterClientScriptResource(this.GetType(), JSResource);
    base.RegisterScriptModules(page);
}
```

To Add Custom Controls to the Widget Properties Dialog Box

The **Widget Properties** dialog box is displayed when a user creates or edits a widget. If you need to add custom controls, such as buttons or grids, to this dialog box, you need to create these controls in the `RenderSettings()` or `RenderSettingsComplete()` method of the widget class, as is described in the sections below.

To Add Buttons to the Widget Properties Dialog Box Dynamically

If you need to add buttons to the **Widget Properties** dialog box that appear based on a particular user action in the dialog box, override the `RenderSettings()` method of the widget class so that it dynamically adds the needed controls to the dialog box. The method must return `true` if all controls are created in the method implementation (that is, no automatic generation of controls is required). The default implementation of the `RenderSettings()` method of the `PXWidgetBase` class returns `false`.

The following example provides the implementation of this method in the predefined Power BI tile widget. When a user clicks the **Sign In** button in the **Widget Properties** dialog box and successfully logs in to Microsoft Entra ID, the **Dashboard** and **Tile** boxes appear in the dialog box.

```
public override bool RenderSettings(PXDataSource ds, WebControl owner)
{
```

```
    var cc = owner.Controls;
    var btn = new PXButton() { ID = "btnAzureLogin", Text =
      PXLocalizer.Localize(Messages.PowerBISignIn, typeof(Messages).FullName),
      Width = Unit.Pixel(100) };
    btn.ClientEvents.Click = "PowerBIWidget.authorizeButtonClick";

    cc.Add(new PXLayoutRule() { StartColumn = true, ControlSize = "XM",
    LabelsWidth = "SM" });
    cc.Add(new PXTextEdit() { DataField = "ClientID", CommitChanges = true });
    cc.Add(new PXLayoutRule() { Merge = true });
    cc.Add(new PXTextEdit() { DataField = "ClientSecret",
      CommitChanges = true });
    cc.Add(btn);
    cc.Add(new PXLayoutRule() { });
    cc.Add(new PXDropDown() { DataField = "DashboardID",
      CommitChanges = true });
    cc.Add(new PXDropDown() { DataField = "TileID", CommitChanges = true });
    cc.Add(new PXTextEdit() { DataField = "AccessCode",
      CommitChanges = true });
    cc.Add(new PXTextEdit() { DataField = "RedirectUri" });
    cc.Add(new PXTextEdit() { DataField = "AccessToken" });
    cc.Add(new PXTextEdit() { DataField = "RefreshToken" });

    foreach (Control wc in cc)
    {
        IFieldEditor fe = wc as IFieldEditor;
        if (fe != null) wc.ID = fe.DataField;
        wc.ApplyStyleSheetSkin(ds.Page);

        PXTextEdit te = wc as PXTextEdit;
        if (te != null) switch (te.ID)
            {
                case "ClientID":
                    te.ClientEvents.Initialize =
                      "PowerBIWidget.initializeClientID";
                    break;
                case "ClientSecret":
                    te.ClientEvents.Initialize =
                      "PowerBIWidget.initializeClientSecret";
                    break;
                case "AccessCode":
                    te.ClientEvents.Initialize =
                      "PowerBIWidget.initializeAccessCode";
                    break;
                case "RedirectUri":
                    te.ClientEvents.Initialize =
                      "PowerBIWidget.initializeRedirectUri";
                    break;
            }
    }
    return true;
}
```

To Open a Pop-Up Panel in the Widget Properties Dialog Box

If you need to open a pop-up panel in the **Widget Properties** dialog box, override the `RenderSettingsComplete()` method of the widget class and create the panel within it.

The following code shows a sample implementation of the method in the predefined chart widget. The method adds the buttons to the dialog box and creates the pop-up panel after the standard controls of the dialog box have been created.

```
public override void RenderSettingsComplete(PXDataSource ds, WebControl owner)
{
    var btn = _btnConfig = new PXButton() {
      ID = "btnConfig", Width = Unit.Pixel(150),
      Text = PXLocalizer.Localize(Messages.ChartConfigure,
        typeof(Messages).FullName),
      PopupPanel = "pnlConfig", Enabled = false, CallbackUpdatable = true };
    owner.Controls.Add(btn);
    btn.ApplyStyleSheetSkin(owner.Page);

    owner.Controls.Add(CreateSettingsPanel(ds, ds.PrimaryView));
    (ds.DataGraph as ChartSettingsMaint).InquiryIDChanged += (s, e) =>
      _btnConfig.Enabled = !string.IsNullOrEmpty(e);
    base.RenderSettingsComplete(ds, owner);
}
```

Customizing Business Events in Code

In Acumatica ERP, various business processes are executed, which may require the monitoring of particular activities and conditions in the system. To eliminate the need for a user to monitor these business processes, you can configure the system to monitor the company data and to perform an action (such as sending an email notification or performing the instructions defined by an import scenario) or multiple actions in the system.

To configure the system to monitor a business process, on the *Business Events* (SM302050) form, you define a *business event* that relates to this business process and that causes the system to perform an action or multiple actions in the system. To define the actions that the system should perform in the system once the business event has occurred, you specify the *subscribers* of this business event on the **Subscribers** tab of the *Business Events* form. You can define custom subscribers of business events in addition to the subscribers available in the system, as described in this chapter. For more information on business events, see *Business Events: General Information*.

To Define a Custom Subscriber Type for Business Events

To define the actions that the system should perform once a business event has occurred, you specify the *subscribers* of this business event on the **Subscribers** tab of the *Business Events* (SM302050) form. A subscriber of a business event is an entity that the system processes when the business event occurs. The subscriber types available in the system include import scenarios, email notifications, mobile push notifications, and mobile SMS notifications. You can define a custom subscriber type for business events, as described in this topic.

> A custom subscriber type can be implemented in a project of your Acumatica ERP extension library. You cannot include the custom subscriber type in a *Code* item of a customization project.

For more information on business events, see *Business Events: General Information*.

Creation of a Custom Subscriber Type for Business Events

1. Define a class that implements the `PX.BusinessProcess.Subscribers.ActionHandlers.IEventAction` interface, which is a subscriber that the system executes once the business event has occurred.
2. In the class that implements the `IEventAction` interface, implement the following methods and properties of the interface:
   - The `Id` property, which is the GUID that identifies the subscriber. For the predefined subscriber types, the system assigns the value of this property to a new subscriber created by a user. The property uses the following syntax.

     ```
     Guid Id { get; set; }
     ```

   - The `Name` property, which is the name of the subscriber of the custom type. For the predefined subscriber types, a user specifies the value of this property on the form that corresponds to the subscriber. For example, for email notifications, the user specifies the value of this property in the **Description** box on the *Email Templates* (SM204003) form. Use the following syntax for the property.

     ```
     string Name { get; }
     ```

   - The `Process` method, which implements the actions that the system should perform once the business event has occurred. For example, for email notifications, the method inserts values in the notification template and sends the notification. The method uses the following syntax.

     ```
     void Process(MatchedRow[] eventRows, CancellationToken cancellation);
     ```

3. Define a class that implements the `PX.BusinessProcess.Subscribers.Factories.IBPSubscriberActionHandlerFactory` interface, which creates and executes the subscriber.
4. In the class that implements the `IBPSubscriberActionHandlerFactory` interface, implement the following methods and properties of the interface:
   - The `CreateActionHandler` method, which creates a subscriber with the specified ID. Use the following syntax for the method.

     ```
     IEventAction CreateActionHandler(Guid handlerId, bool stopOnError,
                IEventDefinitionsProvider eventDefinitionsProvider);
     ```

   - The `GetHandlers` method, which retrieves the list of subscribers of the custom type. This list is displayed in the lookup dialog box in the **Subscriber ID** column on the **Subscribers** tab of the *Business Events* (SM302050) form. The method uses the following syntax.

     ```
     IEnumerable<BPHandler> GetHandlers(PXGraph graph);
     ```

   - The `RedirectToHandler` method, which performs redirection to the subscriber. For example, for email notifications, the method opens the *Email Templates* form, which displays the subscriber (which is a notification template) with the specified ID. Use the following syntax for the method.

     ```
     void RedirectToHandler(Guid? handlerId);
     ```

   - The `Type` property, which is a `string` identifier of the subscriber type that is exactly four characters long. The value of this property is stored in the database. The property uses the following syntax.

     ```
     string Type { get; }
     ```

   - The `TypeDescription` property, which is a `string` label of the subscriber type. A user views this value in the **Type** column on the **Subscribers** tab of the *Business Events* form. Use the following syntax for the property.

     ```
     string TypeDescription { get; }
     ```

   > If you want an action to be displayed in the **Create Subscriber** menu on the toolbar of the **Subscribers** tab of the *Business Events* (SM302050) form, instead of implementing the `IBPSubscriberActionHandlerFactory` interface, implement the `IBPSubscriberActionHandlerFactoryWithCreateAction` interface, which also provides methods and properties that define the creation action.

5. Compile your Acumatica ERP extension library with the implementation of the classes.
6. Open Acumatica ERP and test the new subscriber type.

Example

The following code shows an example of the implementation of a custom subscriber type. This custom subscriber writes the body of the notification to a text file.

```
using System;
using System.Collections.Generic;
using System.Linq;
using PX.BusinessProcess.Subscribers.ActionHandlers;
using PX.BusinessProcess.Subscribers.Factories;
using PX.BusinessProcess.Event;
using PX.BusinessProcess.DAC;
using PX.BusinessProcess.UI;
```

```
using System.Threading;
using PX.Data;
using PX.Common;
using PX.SM;
using System.IO;
using PX.Data.Wiki.Parser;
using PX.PushNotifications;

namespace CustomSubscriber
{
    //The custom subscriber that the system executes once the business event
    //has occurred
    public class CustomEventAction : IEventAction
    {
        //The GUID that identifies a subscriber
        public Guid Id { get; set; }

        //The name of the subscriber of the custom type
        public string Name { get; protected set; }

        //The notification template
        private readonly Notification _notificationTemplate;

        //The method that writes the body of the notification to a text file
        //once the business event has occurred
        public void Process(MatchedRow[] eventRows,
        CancellationToken cancellation)
        {
            using (StreamWriter file =
              new StreamWriter(@"C:\tmp\EventRows.txt"))
            {
                var graph = PXGenericInqGrph.CreateInstance(
                  _notificationTemplate.ScreenID);
                var parameters = @eventRows.Select(
                  r => Tuple.Create<IDictionary<string, object>,
                  IDictionary<string, object>>(
                    r.NewRow?.ToDictionary(c => c.Key.FieldName, c => c.Value),
                    r.OldRow?.ToDictionary(c => c.Key.FieldName,
                      c => (c.Value as ValueWithInternal)?.ExternalValue ??
                        c.Value))).ToArray();
                var body = PXTemplateContentParser.ScriptInstance.Process(
                  _notificationTemplate.Body, parameters, graph, null);
                file.WriteLine(body);
            }
        }

        //The CustomEventAction constructor
        public CustomEventAction(Guid id, Notification notification)
        {
            Id = id;
            Name = notification.Name;
            _notificationTemplate = notification;
        }
    }

    //The class that creates and executes the custom subscriber
    class CustomSubscriberHandlerFactory :
```

```
    IBPSubscriberActionHandlerFactoryWithCreateAction
  {
      //The method that creates a subscriber with the specified ID
      public IEventAction CreateActionHandler(Guid handlerId,
        bool stopOnError, IEventDefinitionsProvider eventDefinitionsProvider)
      {
          var graph = PXGraph.CreateInstance<PXGraph>();
          Notification notification = PXSelect<Notification,
            Where<Notification.noteID,
            Equal<Required<Notification.noteID>>>>
              .Select(graph, handlerId).AsEnumerable().SingleOrDefault();

          return new CustomEventAction(handlerId, notification);
      }

      //The method that retrieves the list of subscribers of the custom type
      public IEnumerable<BPHandler> GetHandlers(PXGraph graph)
      {
          return PXSelect<Notification, Where<Notification.screenID,
            Equal<Current<BPEvent.screenID>>,
            Or<Current<BPEvent.screenID>, IsNull>>>
              .Select(graph).FirstTableItems.Where(c => c != null)
              .Select(c => new BPHandler { Id = c.NoteID, Name = c.Name,
                Type = LocalizableMessages.CustomNotification });
      }

      //The method that performs redirection to the subscriber
      public void RedirectToHandler(Guid? handlerId)
      {
          var notificationMaint =
            PXGraph.CreateInstance<SMNotificationMaint>();
          notificationMaint.Message.Current =
            notificationMaint.Notifications.
              Search<Notification.noteID>(handlerId);
          PXRedirectHelper.TryRedirect(notificationMaint,
            PXRedirectHelper.WindowMode.New);
      }

      //A string identifier of the subscriber type that is
      //exactly four characters long
      public string Type
      {
          get { return "CTTP"; }
      }

      //A string label of the subscriber type
      public string TypeName
      {
          get { return LocalizableMessages.CustomNotification; }
      }

      //A string identifier of the action that creates
      //a subscriber of the custom type
      public string CreateActionName
      {
          get { return "NewCustomNotification"; }
      }
```

```
        //A string label of the button that creates
        //a subscriber of the custom type
        public string CreateActionLabel
        {
            get { return LocalizableMessages.CreateCustomNotification; }
        }

        //The delegate for the action that creates
        //a subscriber of the custom type
        public Tuple<PXButtonDelegate, PXEventSubscriberAttribute[]>
          getCreateActionDelegate(BusinessProcessEventMaint maintGraph)
        {
            PXButtonDelegate handler = (PXAdapter adapter) =>
            {
                if (maintGraph.Events?.Current?.ScreenID == null)
                    return adapter.Get();

                var graph = PXGraph.CreateInstance<SMNotificationMaint>();
                var cache = graph.Caches<Notification>();
                var notification = (Notification)cache.CreateInstance();
                var row = cache.InitNewRow(notification);
                row.ScreenID = maintGraph.Events.Current.ScreenID;
                cache.Insert(row);

                var subscriber = new BPEventSubscriber();
                var subscriberRow =
                  maintGraph.Subscribers.Cache.InitNewRow(subscriber);
                subscriberRow.Type = Type;
                subscriberRow.HandlerID = row.NoteID;
                graph.Caches[typeof(BPEventSubscriber)].Insert(subscriberRow);

                PXRedirectHelper.TryRedirect(graph,
                  PXRedirectHelper.WindowMode.NewWindow);
                return adapter.Get();
            };
            return Tuple.Create(handler,
                new PXEventSubscriberAttribute[]
                    {new PXButtonAttribute {
                      OnClosingPopup = PXSpecialButtonType.Refresh}});
        }
    }

    //Localizable messages
    [PXLocalizable]
    public static class LocalizableMessages
    {
        public const string CustomNotification = "Custom Notification";
        public const string CreateCustomNotification = "Custom Notification";
    }
}
```

To test this example, do the following:

1. Add this code to an Acumatica ERP extension library. For details on creating the library, see *To Create an Extension Library*.
2. Make sure that the following references are added to the project of the extension library:

- `PX.Data`
- `PX.Common`
- `PX.Common.Std`
- `PX.BusinessProcess`

3. Create a business event on the *Business Events* (SM302050) form. For details on the creation of a business event, see *Business Events: General Information*.
4. On the **Subscribers** tab, make sure *Custom Notification* is available in the **Type** column.
5. Select *Custom Notification* in the **Type** column. Make sure the list in the **Subscriber ID** column is empty.
6. On the table toolbar, make sure **Create Subscriber > Custom Notification** is available.
7. Click **Create Subscriber > Custom Notification**. Make sure the *Email Templates* (SM204003) form opens.
8. On the *Email Templates* form, create a notification template with a message that is not empty on the **Messages** tab. For details about notification templates, see *Email Templates*.
9. Make sure the created notification template is added to the **Subscribers** tab of the *Business Events* form as the *Custom Notification* subscriber and save the business event.
10. Make changes in the system to trigger the business event that you have configured.
11. Make sure the text file (in this example, `C:\tmp\EventRows.txt`) contains the body of the notification.

Related Links

- *Business Events: General Information*

Adding a Custom SMS Provider

In Acumatica ERP, you can register a short message service (SMS) account from which the system will send text notifications. The Twilio and Amazon SMS providers are available by default. You can also define and register a custom SMS provider.

Custom SMS Provider: General Information

To send SMS notifications, the system uses SMS providers, which are available on the *SMS Providers* (SM203535) form. By default, the system includes the Twilio and Amazon SMS providers. You can add a custom SMS provider by using the platform API.

Learning Objectives

In this chapter, you will learn how to do the following:

- Create a custom SMS provider
- Register the custom SMS provider in the system

Applicable Scenarios

You add a custom SMS provider in the following cases:

- Your company has a business relationship with an SMS provider that is not available by default, and you need to support two-factor authentication through SMS notifications sent by this provider. For details about the use of SMS notifications for two-factor authentication, see *Two-Factor Authentication: General Information*.
- Your company has a business relationship with an SMS provider that is not available by default and you need to support notifications about business events through SMS sent by this provider. For details about SMS notifications for business events, see *Business Events: Subscribers*.

Implementation of a Custom SMS Provider

Acumatica ERP provides a set of interfaces for the implementation of SMS providers. To add a custom SMS provider, you need to implement the `PX.SmsProvider.ISmsProvider` and `PX.SmsProvider.ISmsProviderFactory` interfaces.

On the *SMS Providers* (SM203535) form, Acumatica ERP automatically discovers each class that implements the `PX.SmsProvider.ISmsProvider` interface and displays the SMS provider type that corresponds to the class in the **Provider Type** box.

> A custom SMS provider can be implemented in a project of your Acumatica ERP extension library. You cannot include the custom SMS provider in a *Code* item of a customization project.

ISmsProvider Interface

The class that implements the `PX.SmsProvider.ISmsProvider` interface defines the parameters of the SMS provider and the logic for SMS processing.

In this class, you implement the following elements of the `ISmsProvider` interface:

- `ExportSettings`: You use this property to define the list of parameters that are displayed on the **Parameters** tab of the *SMS Providers* (SM203535) form.
- `LoadSettings`: By using this property, you obtain the values that a user has assigned to the parameters on the **Parameters** tab and assign them to variables in the class.
- `SendMessageAsync`: You use this method to implement the logic of SMS processing.

ISmsProviderFactory Interface

The class that implements the *`PX.SmsProvider.ISmsProviderFactory`* interface is a factory class for the initialization of the SMS provider.

In this class, you implement the following elements of the `ISmsProviderFactory` interface:

- `Create`: You define two methods for the creation of the SMS provider, one without parameters and another with the specified list of parameters, which are displayed on the **Parameters** tab of the *SMS Providers* (SM203535) form.
- `Description`: You use this property to specify the description of the SMS provider, such as *Plivo SMS provider*, to be displayed in the **Description** column of the **Select - Provider Type** lookup table (which is displayed when a user clicks the magnifier button in the **Provider Type** box on the *SMS Providers* form).
- `Name`: By using this property, you specify the type of the SMS provider, such as *PX.SmsProvider.Plivo.PlivoSmsProvider*, to be displayed in the **Type Name** column of the **Select - Provider Type** lookup table (which is displayed when a user clicks the magnifier button in the **Provider Type** box on the *SMS Providers* form).

Custom SMS Provider: To Add a Custom SMS Provider

The following activity will walk you through the process of adding a custom SMS provider.

Story

Suppose that your company has a business relationship with the Plivo SMS provider, which is not supported by default by Acumatica ERP. You need to support the sending of SMS notifications through this SMS provider in Acumatica ERP.

Process Overview

You will implement the *`PX.SmsProvider.ISmsProvider`* and *`PX.SmsProvider.ISmsProviderFactory`* interfaces in the Visual Studio project of the extension library. You will then publish the customization project with the implemented interfaces and test the availability of the provider on the *SMS Providers* (SM203535) form.

System Preparation

Before you begin performing the steps of this activity, do the following:

1. Prepare an Acumatica ERP instance, as described in *Instance Deployment: To Deploy an Instance with Demo Data*.
2. Create a new customization project. For details about this process, see *Customization Projects: To Create a Customization Project*.
3. Create an extension library, as described in *To Create an Extension Library*.

You can find the source code of this activity in the *Help-and-Training-Examples* repository on GitHub.

Step 1: Implementing the ISmsProvider Interface

To implement the `PX.SmsProvider.ISmsProvider` interface, do the following:

1. In the Visual Studio project of the extension library, add a reference to the `PX.SmsProvider.Core.dll` library from the `Bin` folder of the Acumatica ERP instance.
2. In the project, define the `PlivoSmsProvider` class, which implements the `ISmsProvider` interface, in the `PX.SmsProvider.Plivo` namespace.

```
using System.Collections.Generic;
using System.Threading;
using System.Threading.Tasks;
using PX.Data;

namespace PX.SmsProvider.Plivo
{
    public class PlivoSmsProvider : ISmsProvider
    {
    }
}
```

3. In the project, add the `Helper` folder. Within the folder, add the `Messages.cs` file and add the following code to it.

```
using PX.Common;

namespace PX.SmsProvider.Plivo
{
    [PXLocalizable]
    public static class Messages
    {
        public const string AuthID_DetailID_Display = "Auth ID";
        public const string AuthToken_DetailID_Display = "Auth Token";
        public const string FromPhoneNbr_DetailID_Display = "From Number";
    }
}
```

4. In the `ExportSettings` method of the class, add the following code. This code adds the `Auth ID`, `Auth Token`, and `From Number` parameters to the list that will be displayed on the **Parameters** tab of the *SMS Providers* (SM203535) form. You need these parameters to work with Plivo.

```
using System.Collections.Generic;
using System.Threading;
using System.Threading.Tasks;
using PX.Data;

namespace PX.SmsProvider.Plivo
{
    public class PlivoSmsProvider : ISmsProvider
    {
        #region DetailIDs const
        private const string AuthID_DetailID = "AUTH_ID";
        private const string AuthToken_DetailID = "AUTH_TOKEN";
        private const string FromPhoneNbr_DetailID = "FROM_PHONE_NBR";
```

```
        #endregion

        private string m_AuthID;
        public string AuthID { get { return m_AuthID; } }

        private string m_AuthToken;
        public string AuthToken { get { return m_AuthToken; } }

        private string m_FromPhoneNbr;
        public string FromPhoneNbr { get { return m_FromPhoneNbr; } }

        public IEnumerable<PXFieldState> ExportSettings
        {
            get
            {
                var settings = new List<PXFieldState>();

                var authID = (PXStringState)PXStringState.CreateInstance(
                    m_AuthID,
                    null,
                    false,
                    AuthID_DetailID,
                    null,
                    1,
                    null,
                    null,
                    null,
                    null,
                    null
                );
                authID.DisplayName = Messages.AuthID_DetailID_Display;
                settings.Add(authID);
                var authToken = (PXStringState)PXStringState.CreateInstance(
                    m_AuthToken,
                    null,
                    false,
                    AuthToken_DetailID,
                    null,
                    1,
                    "*",
                    null,
                    null,
                    null,
                    null
                );
                authToken.DisplayName = Messages.AuthToken_DetailID_Display;
                settings.Add(authToken);

                var fromPhoneNbr = (PXStringState)PXStringState.CreateInstance(
                    m_FromPhoneNbr,
                    null,
                    false,
                    FromPhoneNbr_DetailID,
                    null,
                    1,
                    null,
                    null,
```

```
                    null,
                    null,
                    null
                );
                fromPhoneNbr.DisplayName = Messages.FromPhoneNbr_DetailID_Display;
                settings.Add(fromPhoneNbr);

                return settings;
            }
        }
    }
}
```

5. In the `LoadSettings` method of the class, obtain the values that have been entered on the **Parameters** tab of the *SMS Providers* form and assign them to variables in the class, as shown in the following code.

```
public void LoadSettings(IEnumerable<ISmsProviderSetting> settings)
{
    foreach (ISmsProviderSetting detail in settings)
    {
        switch (detail.Name.ToUpper())
        {
            case AuthID_DetailID: m_AuthID = detail.Value; break;
            case AuthToken_DetailID: m_AuthToken = detail.Value; break;
            case FromPhoneNbr_DetailID: m_FromPhoneNbr = detail.Value; break;
        }
    }
}
```

6. In the class, implement SMS processing in the `SendMessageAsync` method. For an example of the implementation of this method, see *PlivoSmsProvider.cs* in the GitHub repository.

Step 2: Implementing the ISmsProviderFactory Interface

Implement the *`PX.SmsProvider.ISmsProviderFactory`* interface as follows:

1. In the `Messages.cs` file, add the name of the provider that will be displayed in the **Provider Type** box of the *SMS Providers* (SM203535) form, as shown in the following code.

```
using PX.Common;

namespace PX.SmsProvider.Plivo
{
    [PXLocalizable]
    public static class Messages
    {
        ...
        public const string ProviderName = "Plivo SMS provider";
    }
}
```

2. Define the `PlivoSmsProviderFactory` class, which implements the `PX.SmsProvider.ISmsProviderFactory` interface, as the following code shows.

```
using System.Collections.Generic;

namespace PX.SmsProvider.Plivo
```

```
{
    public class PlivoSmsProviderFactory : ISmsProviderFactory
    {
        public ISmsProvider Create(IEnumerable<ISmsProviderSetting> settings)
        {
            var provider = new PlivoSmsProvider();
            provider.LoadSettings(settings);
            return provider;
        }

        public ISmsProvider Create()
        {
            var provider = new PlivoSmsProvider();
            return provider;
        }

        public string Description { get; } = Messages.ProviderName;
        public string Name { get; } = typeof(PlivoSmsProvider).FullName;
    }
}
```

Step 3: Testing the SMS Provider

To test the Plivo SMS provider, do the following:

1. Build the Visual Studio project.
2. On the *SMS Providers* (SM203535) form, make sure that the new SMS provider is available for selection in the **Provider Type** box, as shown in the following screenshot.

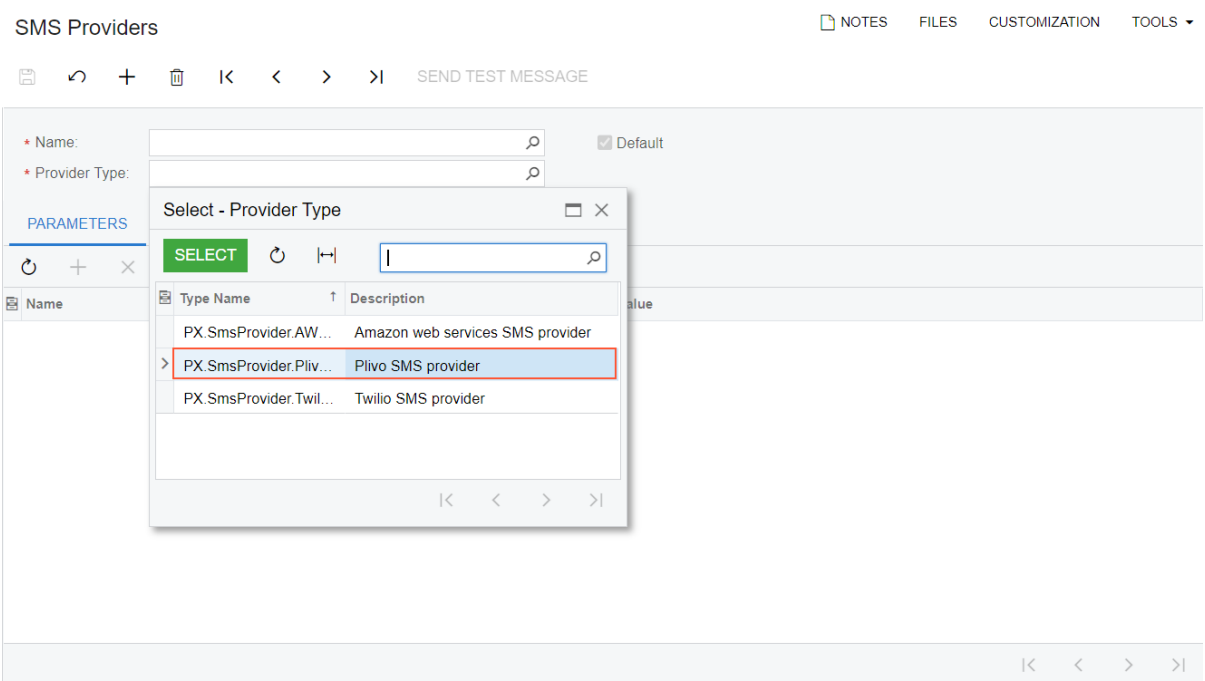


Figure: The custom SMS provider

3. Include the extension library in the customization project as a *File* item, as described in *To Add a Custom File to a Project*.

4. Optional: Test the SMS provider as follows:
   a. Create a Plivo account on *https://console.plivo.com/accounts/register/*, and get your Plivo phone number.
   b. On the *SMS Providers* (SM203535) form, create a new SMS provider by specifying the following parameters:
      a. **Name**: *Plivo SMS Provider*
      b. **Provider Type**: *Plivo SMS Provider (PX.SmsProvider.Plivo.PlivoSmsProvider)*
      c. **Auth ID**: The Auth ID from your account dashboard in Plivo console
      d. **Auth Token**: The Auth Token from your account dashboard in Plivo console
      e. **From Number**: The Plivo phone number from which the message will be sent
   c. Save your changes, and click **Send Test Message** on the form toolbar.

Implementing Plug-Ins for Processing Credit Card Payments

Acumatica ERP processes credit card and debit card payments through Acumatica Payments. To work with the payment gateway, Acumatica ERP supports the *Acumatica Payments Plug-In*. For more information on this plug-in, see *Configuring and Using Acumatica Payments*.

In a customized Acumatica ERP application, you can implement payment plug-ins that work with credit card payment processing centers other than Acumatica Payments. In this chapter, you can find information about how to develop custom plug-ins and use them in Acumatica ERP.

Interfaces for Processing Credit Card Payments

Acumatica ERP provides the interfaces for the implementation of plug-ins for credit card payment processing.

By using these interfaces, you can implement tokenized processing plug-ins. When a tokenized processing plug-in is used, the credit card information is not saved to the application database; this information is stored only at the processing center. The Acumatica ERP database stores only the identification tokens that link customers and payment methods in the application with the credit card data at the processing center.

For details on how to implement custom plug-ins, see *To Implement a Plug-In for Processing Credit Card Payments*.

> In Acumatica ERP, the *Acumatica Payments Plug-In* implements the interfaces described in the sections below. For more information about the built-in plug-in, see *Configuring and Using Acumatica Payments*.

Mandatory Interfaces

The root interface for implementation of custom plug-ins for credit card processing is `PX.CCProcessingBase.Interfaces.V2.ICCProcessingPlugin`. The system automatically discovers the class that implements the `ICCProcessingPlugin` interface in the `Bin` folder of the Acumatica ERP instance and includes it in the list in the **Payment Plug-In (Type)** box on the *Processing Centers* (CA205000) form. For creation of a custom plug-in, you also need to implement the `ICCTransactionProcessor` interface to process credit card transactions.

Additional Interfaces

You can implement the following additional functionality:

- Tokenized credit card processing: Implement the `ICCProfileProcessor` and `ICCHostedFormProcessor` interfaces.
- Processing of payments from new credit cards: To use this functionality, implement the `ICCProfileCreator`, `ICCHostedPaymentFormProcessor`, `ICCHostedPaymentFormResponseParser`, and `ICCTransactionGetter` interfaces. If this functionality is implemented in a custom plug-in, a user can select the **Accept Payment from New Card** check box on the *Processing Centers* form for the processing centers that use this custom plug-in.
- Synchronization of credit cards with the processing center: Implement the `ICCTransactionGetter` interface in a custom plug-in. If this functionality is implemented, on the *Synchronize Cards* (CA206000) form, users can work with the processing centers that use this custom plug-in.
- Retrieval of the information about suspicious credit card transactions (without the use of the hosted form that accepts payments): To use this functionality, implement the `ICCTranStatusGetter` interface.

- Webhooks as a way to obtain a response from the processing center: Implement the `ICCWebhookProcessor` and `ICCWebhookResolver` interfaces.

Related Links

- *To Implement a Plug-In for Processing Credit Card Payments*

To Implement a Plug-In for Processing Credit Card Payments

In Acumatica ERP, the built-in *Acumatica Payments Plug-In* processes transactions in the Acumatica Payments processing center. For more information on this plug-in, see *Configuring and Using Acumatica Payments*. You can implement your own plug-in for working with processing centers other than Acumatica Payments, as described in this topic.

To Implement a Credit Card Processing Plug-In

1. In a class library project, define a class that implements the *`PX.CCProcessingBase.Interfaces.V2.ICCTransactionProcessor`* interface, which provides credit card processing functionality. In the class, implement the `DoTransaction` method, which processes the transaction in the payment gateway.

   The following code shows the declaration of a class that implements the `ICCTransactionProcessor` interface.

   ```
   using PX.CCProcessingBase.Interfaces.V2;

   public class MyTransactionProcessor : ICCTransactionProcessor
   {
     public ProcessingResult DoTransaction(ProcessingInput inputData)
     {
       ...
     }
   }
   ```

2. If you need to implement tokenized processing, define the following classes:
   - A class that implements the *`PX.CCProcessingBase.Interfaces.V2.ICCProfileProcessor`* interface (which provides methods to manage customer and payment profiles)
   - A class that implements the *`PX.CCProcessingBase.Interfaces.V2.ICCHostedFormProcessor`* interface (which provides methods to work with hosted forms for the creation and management of payment profiles)

   In the classes, implement the methods of the interfaces.

3. If you need to implement payment processing without the preliminary creation of payment profiles, define the following classes:
   - A class that implements the *`PX.CCProcessingBase.Interfaces.V2.ICCProfileCreator`* interface (which provides the method to create the payment profile for a new credit card)
   - A class that implements the *`PX.CCProcessingBase.Interfaces.V2.ICCHostedPaymentFormProcessor`* interface (which provides methods to work with hosted forms for processing payments without preliminary creation of payment profiles)
   - A class that implements the *`PX.CCProcessingBase.Interfaces.V2.ICCHostedPaymentFormResponseParser`* interface (which provides a method to parse the response after a successful operation on a hosted form that processes payments without preliminary creation of payment profiles)

- A class that implements the *PX.CCProcessingBase.Interfaces.V2.ICCTransactionGetter* interface (which provides the methods to obtain information about transactions by transaction ID)

In the classes, implement the methods of the interfaces.

4. If you need to implement the retrieval of detailed information about transactions, which can be used for the synchronization of transactions with the processing center, define a class that implements the *PX.CCProcessingBase.Interfaces.V2.ICCTransactionGetter* interface (which provides the methods to obtain information about transactions by transaction ID).
5. If you need to retrieve the information about suspicious credit card transactions with the *Held for Review* status (without the use of the hosted form that accepts payments), define a class that implements the supplementary *PX.CCProcessingBase.Interfaces.V2.ICCTranStatusGetter* interface (which provides the method to obtain the transaction status after the execution of the `ICCTransactionProcessor.DoTransaction` method). In the class, implement the methods of the interface.
6. If you need to support webhooks as a way to obtain a response from the processing center, define the following classes:
   - A class that implements the *PX.CCProcessingBase.Interfaces.V2.ICCWebhookProcessor* interface (which provides the methods to add, update, and delete webhooks and retrieve the list of webhooks)
   - A class that implements the *PX.CCProcessingBase.Interfaces.V2.ICCWebhookResolver* interface (which parses the information that comes from the processing center through webhooks)

   In the classes, implement the methods of the interfaces.
7. Define a class that implements the `PX.CCProcessingBase.Interfaces.V2.ICCProcessingPlugin` interface, which is the root interface for credit card payment processing and is used by Acumatica ERP to find the plug-in in the application libraries. The class should have a public parameterless constructor (either explicit or default). In the class, implement the methods of the *ICCProcessingPlugin* interface as follows:
   - In the `ExportSettings` method, which exports the required settings from the plug-in to the *Processing Centers* (CA205000) form, return a collection that contains the settings that can be configured by the user on the form. The syntax of the method is shown in the following code.

     ```
     IEnumerable<SettingsDetail> ExportSettings();
     ```

   - In the `ValidateSettings` method, validate the settings modified on the *Processing Centers* form, which are passed as the parameter of the method, and return `null` if validation was successful or an error message if validation failed. The syntax of the method is shown in the following code.

     ```
     string ValidateSettings(SettingsValue setting);
     ```

   - In the `TestCredentials` method, check the connection to the payment gateway by using the credentials that are specified by the user on the *Processing Centers* form. The syntax of the method is shown in the following code.

     ```
     void TestCredentials(IEnumerable<SettingsValue> settingValues);
     ```

   - In the `CreateProcessor<T>` method, return a new object of the `T` type, and initialize the object with the settings passed to the method. The `T` type can be any of the following:
     - `ICCTransactionProcessor`
     - `ICCProfileProcessor`
     - `ICCHostedFormProcessor`
     - `ICCProfileCreator`
     - `ICCHostedPaymentFormProcessor`
     - `ICCTransactionGetter`

- `ICCTranStatusGetter`
- `ICCWebhookResolver`
- `ICCWebhookProcessor`

If a particular `T` type is not supported by your plug-in, return `null` for this type. The following code shows a sample implementation of the method.

```
public T CreateProcessor<T>(IEnumerable<SettingsValue> settingValues)
  where T : class
{
  if (typeof(T) == typeof(ICCProfileProcessor))
  {
    return new MyProfileProcessor(settingValues) as T;
  }
  if (typeof(T) == typeof(ICCHostedFormProcessor))
  {
    return new MyHostedFormProcessor(settingValues) as T;
  }
  if (typeof(T) == typeof(ICCHostedPaymentFormProcessor))
  {
    return new MyHostedPaymentFormProcessor(settingValues) as T;
  }
  if (typeof(T) == typeof(ICCHostedPaymentFormResponseParser))
  {
    return new MyHostedPaymentFormResponseParser(settingValues) as T;
  }
  if (typeof(T) == typeof(ICCTransactionProcessor))
  {
    return new MyTransactionProcessor(settingValues) as T;
  }
  if (typeof(T) == typeof(ICCTransactionGetter))
  {
    return new MyTransactionGetter(settingValues) as T;
  }
  if (typeof(T) == typeof(ICCProfileCreator))
  {
    return new MyProfileCreator(settingValues) as T;
  }
  if (typeof(T) == typeof(ICCWebhookResolver))
  {
    return new MyWebhookResolver() as T;
  }
  if (typeof(T) == typeof(ICCWebhookProcessor))
  {
    return new MyWebhookProcessor(settingValues) as T;
  }
  if (typeof(T) == typeof(ICCTranStatusGetter))
  {
    return new MyTranStatusGetter() as T;
  }
  return null;
}
```

8. Build your project.
9. To add your plug-in to Acumatica ERP, include the assembly in the customization project. (When the customization project is published, the assembly is copied to the `Bin` folder of the Acumatica ERP website automatically.) During startup, the system automatically discovers the class that implements the

`ICCProcessingPlugin` interface and includes it in the list in the **Payment Plug-In (Type)** box on the *Processing Centers* form.

Related Links

- *Interfaces for Processing Credit Card Payments*
- *PX.CCProcessingBase.Interfaces.V2 Namespace*

Implementing a Connector for an External System

Acumatica ERP provides built-in integrations with the BigCommerce, Shopify, and Amazon ecommerce systems. For details about these integrations, see *Integration with BigCommerce* and *Integration with Shopify*. Each integration uses the Acumatica Commerce Framework to implement the connector between Acumatica ERP and the external system.

By using the Acumatica Commerce Framework, you can implement a custom connector for an external system that provides REST API. In this chapter, you can find information about how to develop a custom connector and use it in Acumatica ERP.

Connector Implementation: General Information

A connector between Acumatica ERP and an external system is a plug-in that synchronizes particular entities in Acumatica ERP with the corresponding entities in the external system. This synchronization can be performed based on a schedule or in real time.

Learning Objectives

In this chapter, you will learn how to create a custom connector.

Applicable Scenarios

You create a connector between Acumatica ERP and an external system if you need to synchronize data between Acumatica ERP and the external system.

Parts of the Connector

The connector consists of the following major parts, which are shown in the diagram below:

- Classes for Acumatica ERP entities, which function as adapters for the entities of the contract-based REST API of Acumatica ERP. For more information about these classes, see *Connector Implementation: Classes for Acumatica ERP Entities*.
- Classes for external entities, which function as adapters for the entities of the API of the external system. For details about these classes, see *Connector Implementation: Classes for External Entities*.
- Mapping classes, which define the mappings between internal and external entities. Mapping classes are described in *Connector Implementation: Mapping Classes*.
- Bucket classes, which define buckets of entities whose synchronization depends on one another. For more information, see *Connector Implementation: Bucket Classes*.
- Processor classes, which implement the synchronization of the entities between the external system and Acumatica ERP. Processor classes are described in greater detail in *Connector Implementation: Processor Classes*.
- A connector class, which connects other classes in the connector code with Acumatica ERP forms and defines the settings for these forms. For details about the implementation of the connector class, see *Connector Implementation: Connector Class*.
- A connector's factory class, which initializes the connector. For details about the connector's factory class, see *Connector Implementation: Connector's Factory Class*.
- A configuration form where an administrator can specify the basic settings of the connector. For example, for the built-in BigCommerce connector, Acumatica ERP provides the *BigCommerce Stores* (BC201000) configuration form.

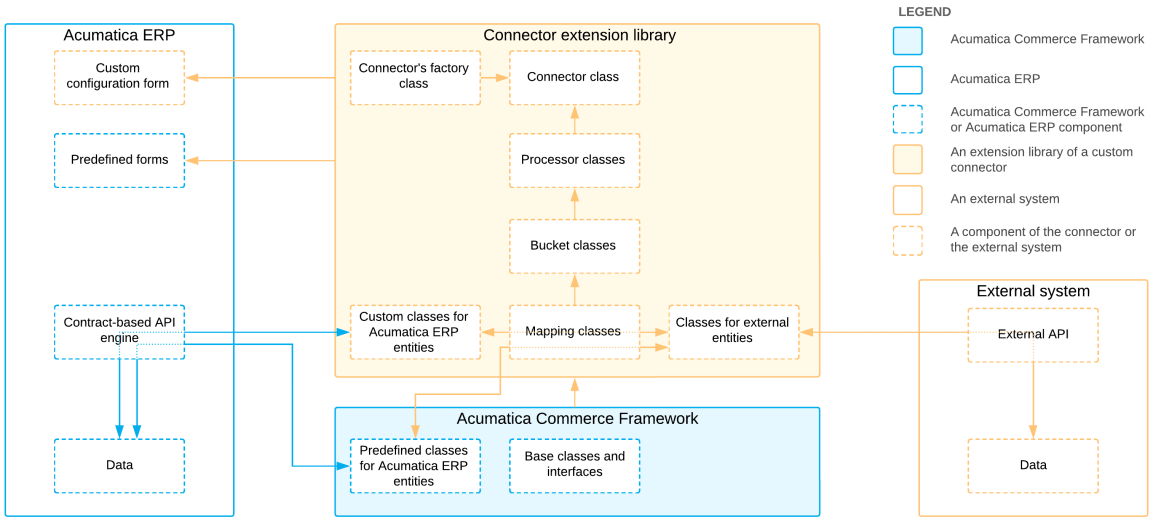


Figure: Connector architecture

Creation of a Connector for an External System

To create a connector for an external system, you need to perform the following general steps:

1. **Creating an extension library for Acumatica ERP.**

   Before you develop the connector for the external system, you need to create an extension library and include it in an Acumatica ERP customization project. For details about the creation of an extension library, see *To Create an Extension Library*.

   Acumatica Customization Platform adds the default references to the project of the extension library, such as `PX.Data` and `PX.Common`. You also need to add the libraries that are described in *Connector Implementation: Required Libraries* to the project of the extension library.

2. **Creating classes for Acumatica ERP entities.**

   You need to create classes for the Acumatica ERP entities to be synchronized with the external system through the connector. For details about the classes, see *Connector Implementation: Classes for Acumatica ERP Entities*.

3. **Creating classes for external entities.**

   You need to create classes for the external entities that you need to synchronize with the Acumatica ERP system through the connector. For details about the classes, see *Connector Implementation: Classes for External Entities*.

4. **Defining the mapping classes between the internal and external entities.**

   For each pair of an internal entity and an external entity that should be synchronized, you must create a mapping class. For details about the mapping classes, see *Connector Implementation: Mapping Classes*.

5. **Creating the bucket classes for the mapped entities.**

   For each mapping class, you need to create a bucket class. For details about bucket classes, see *Connector Implementation: Bucket Classes*.

6. **Creating a DAC with the connector's configuration settings.**

   You need to create a DAC with the configuration settings that will be used by the connector. For an example of the DAC, see *Connector Implementation: DAC for the Connector's Configuration Settings*. For this DAC, you also need to add a database table and include it in the customization project.

7. **Implementing an API client of the external system.**

You need to implement an API client of the external system. For more information about implementation of an API client, see *Connector Implementation: API Client*.

8. **Implementing the processor classes.**

   For each pair of entities that you need to synchronize, you need to create a processor class. For details about processor classes, see *Connector Implementation: Processor Classes*.

9. **Implementing the connector class.**

   In the connector class, you need to implement the synchronization of the Acumatica ERP entities and the external entities. For a detailed description of the connector class, see *Connector Implementation: Connector Class*.

10. **Implementing the connector's factory class.**

    For the system to create the connector class, you need to implement the connector's factory class. For details about the connector factory class, see *Connector Implementation: Connector's Factory Class*.

11. **Creating the configuration form.**

    For a user to specify the basic settings of the connection with the external system, you need to create a custom form in Acumatica ERP. You can create the form in either of the following ways:

    - From scratch. For details about the creation of custom forms, review the *T200 Maintenance Forms* training course.
    - By using the configuration forms that are available for the predefined connectors as a template and adjusting these forms.

12. **Testing the connector.**

    When you have completed the development of the connector, you need to test the connector. To start the testing of the connector, you need to enable the *Custom Connectors* feature on the *Enable/Disable Features* (CS100000) form. (The corresponding check box can be found under **Third Party Integrations > Advanced Integration Engine** on this form.)

Connector Implementation: Required Libraries

The Acumatica Customization Platform adds the default references to the project of the extension library, such as `PX.Data` and `PX.Common`. During the creation of the connector for the external system, you also need to add the references to the assemblies located in the `Bin` folder of the Acumatica ERP instance and to the external assemblies.

Libraries of the Bin Folder

You need to add the references to the following assemblies located in the `Bin` folder of the Acumatica ERP instance:

- `PX.Commerce.Core.dll`: Required for the implementation of any commerce connector
- `PX.Commerce.Objects.dll`: Required for the implementation of any commerce connector
- `PX.Commerce.BigCommerce.dll`: Necessary only if you want to use particular features implemented for the BigCommerce connector
- `PX.Api.ContractBased.dll`: Necessary only if you plan to implement custom classes for Acumatica ERP entities
- `PX.Data.BQL.Fluent.dll`: Necessary only if you plan to use fluent BQL queries instead of traditional BQL queries

External Libraries

You need to add references to the following external libraries:

> You need to use the same version of the library as the one located in the `Bin` folder.

- `RestSharp`
- `Microsoft.Bcl.AsyncIntefaces`
- `Newtonsoft.Json`
- `Serilog`
- `System.Threading.Tasks.Extensions`
- `Polly.Core` (necessary only if you plan to implement the processing of API rate limits, as described in *Processing of API Rate Limits*)

Connector Implementation: Classes for Acumatica ERP Entities

You need to create a class for each Acumatica ERP entity to be synchronized with the external system through the connector. Classes for Acumatica ERP entities are adapters for the entities of the contract-based REST API of Acumatica ERP.

To implement these classes, you have two options: define the classes on your own, or use the classes defined for the predefined ecommerce integrations, which are available in the `PX.Commerce.Core.API` namespace. For example, if you need to work with customers in Acumatica ERP, you can use the predefined `PX.Commerce.Core.API.Customer` class. If the entity is not implemented in the `PX.Commerce.Core.API` namespace, you can implement a custom class for this entity.

Base Class and Interface

The classes for Acumatica ERP entities derive from the *`PX.Commerce.Core.API.CBAPIEntity`* base class, which implements the *`PX.Commerce.Core.ILocalEntity`* interface. The `CBAPIEntity` class includes the set of standard properties of a contract-based API entity, such as `Id`, `RowNumber`, `Delete`, and `ReturnBehavior`. This class also defines a set of properties that are necessary for the synchronization of the entity with an entity from an external system, such as `SyncID` or `SyncTime`.

Properties of Each Custom Class

In each custom class, you define only the properties of the contract-based API entity that you need to use, including the key fields of the entity. The names of the class and its properties should be exactly the same as the names of the corresponding contract-based API entity and its properties in the respective version of the *eCommerce/24.200.001* endpoint.

> If the API defined in the *eCommerce/24.200.001* endpoint is not sufficient for the implementation of your connector, you can create a custom endpoint and use it for your connector. For details about the creation of a custom endpoint, see *To Create a Custom Endpoint* in the Integration Development Guide. To use the custom endpoint for your connector, you need to replace the `PXDefault` attribute of the `WebServiceEndpoint` and `WebServiceVersion` fields of *`BCBinding`* by using the corresponding `CacheAttached` event handlers in the graph of the configuration form. For details about the replacement, see *CacheAttached: General Information*.

You assign the `PX.Commerce.Core.CommerceDescription` attributes with the names of the entity and its properties to the class and its properties, respectively. These names are used as the names of the Acumatica ERP objects and fields on the mapping and filtering tabs of the *Entities* (BC202000) form.

Creation of Classes for Acumatica ERP Entities

To create classes for the Acumatica ERP entities to be synchronized with the external system through the connector, you generally do the following:

1. On the *Web Service Endpoints* (SM207060) form, you open the *eCommerce/24.200.001* endpoint and identify the entities and their fields, including the key fields, that you need to use. For example, for the `Case` entity, you may need to use the `CaseID`, `Subject`, `Description`, `NoteID`, and `LastModifiedDateTime` fields.
2. You identify the predefined classes for Acumatica ERP entities (which are available in the `PX.Commerce.Core.API` namespace) that you can use in your connector.
3. If you need to implement your own classes for Acumatica ERP entities, in the Visual Studio project of the extension library, for each Acumatica ERP entity that you need to use, you create a class with the name of an Acumatica ERP entity that you want to process in the connector.

Example

The following code shows an example of the implementation of a class derived from the `CBAPIEntity`.

> You can see this code on *GitHub*.

```
using PX.Commerce.Core.API;
using PX.Api.ContractBased.Models;
using PX.Commerce.Core;

[CommerceDescription("Case")]
public partial class Case : CBAPIEntity
{
 public GuidValue? NoteID { get; set; }

 public DateTimeValue? LastModifiedDateTime { get; set; }

 [CommerceDescription("CaseCD")]
 public StringValue? CaseCD { get; set; }

 [CommerceDescription("Subject")]
 public StringValue? Subject { get; set; }

 [CommerceDescription("Description")]
 public StringValue? Description { get; set; }
}
```

Connector Implementation: Classes for External Entities

You need to create a class for each external entity to be synchronized with Acumatica ERP through the connector. Classes for external entities are adapters for the entities of the API of the external system.

Base Class and Interface

You define a class that derives from the `PX.Commerce.Core.BCAPIEntity` base class, which implements the `PX.Commerce.Core.IExternEntity` interface. In this class, you define the properties of the external API entity that you need to synchronize with the properties of the entity stored in Acumatica ERP.

Attributes of the Class and Its Properties

You assign the `PX.Commerce.Core.CommerceDescription` attribute with the name of the entity to the class. Entity names are used as the names of the external objects on the *Entities* (BC202000) form.

You also assign the `CommerceDescription` attributes to the properties of the class. In each attribute, you specify the following parameters:

- The name of the property, which is used as the name of the external field on the mapping and filtering tabs of the *Entities* form.
- Optional: A `FieldFilterStatus` value, which specifies whether the field is used on the filtering tabs of the *Entities* form.
- Optional: A `FieldMappingStatus` value, which indicates whether the field is used on the mapping tabs of the *Entities* form.
- Optional: A `String` value that specifies the feature on which the field depends, such as `"PX.Objects.CS.FeaturesSet+userDefinedOrderTypes"`. The field is available on the *Entities* form if the specified feature is enabled on the *Enable/Disable Features* (CS100000) form.

You also assign the `Newtonsoft.Json.JsonProperty` attribute to the properties of the class to map them to the properties of the external entity retrieved in JSON format through the external API.

> The external system determines which classes you need to implement and how the connector communicates with the external system.

Example

The following code shows an example of the declaration of a customer entity for the WooCommerce connector.

> You can see this code on *GitHub*.

```
using System;
using System.Collections.Generic;
using PX.Commerce.Core;
using Newtonsoft.Json;

namespace WooCommerceTest
{
    [CommerceDescription(WCCaptions.Customer)]
    public class CustomerData : BCAPIEntity, IWooEntity
    {
        [JsonProperty("id")]
        [CommerceDescription(WCCaptions.ID, FieldFilterStatus.Skipped,
            FieldMappingStatus.Import)]
        public int? Id { get; set; }

        [JsonProperty("date_created")]
        public DateTime? DateCreatedUT { get; set; }
```

```
        [CommerceDescription(WCCaptions.DateCreatedUT)]
        [ShouldNotSerialize]
        public virtual DateTime? CreatedDateTime
        {
            get
            {
                return DateCreatedUT != null ?
                    DateCreatedUT.ToDate() : default;
            }
        }

        [JsonProperty("date_modified_gmt")]
        public DateTime? DateModified { get; set; }

        [CommerceDescription(WCCaptions.DateModifiedUT)]
        [ShouldNotSerialize]
        public virtual DateTime? ModifiedDateTime
        {
            get
            {
                return DateModified != null ?
                    DateModified.ToDate() : default;
            }
        }

        [JsonProperty("email")]
        [CommerceDescription(WCCaptions.Email, FieldFilterStatus.Skipped,
            FieldMappingStatus.Import)]
        [ValidateRequired]
        public string? Email { get; set; }

        [JsonProperty("first_name")]
        [CommerceDescription(WCCaptions.FirstName, FieldFilterStatus.Skipped,
            FieldMappingStatus.Import)]
        [ValidateRequired]
        public string? FirstName { get; set; }

        [JsonProperty("last_name")]
        [CommerceDescription(WCCaptions.LastName, FieldFilterStatus.Skipped,
            FieldMappingStatus.Import)]
        [ValidateRequired()]
        public string? LastName { get; set; }

        [JsonProperty("username")]
        [CommerceDescription(WCCaptions.UserName, FieldFilterStatus.Skipped,
            FieldMappingStatus.Import)]
        public string? Username { get; set; }

        [JsonProperty("billing")]
        public CustomerAddressData? Billing { get; set; }

        [JsonProperty("shipping")]
        public CustomerAddressData? Shipping { get; set; }
    }
}
```

Connector Implementation: Mapping Classes

For each pair of an internal entity and an external entity that should be synchronized, you need to implement a mapping class, which defines the correspondence between the properties of the internal and external entities.

Base Class and Interfaces

The mapping class implements the following interfaces:

- `PX.Commerce.Core.IMappedEntity`: Represents the mapping between the properties of the local and external objects
- *`PX.Commerce.Core.IMappedEntityExtern<ExternType>`*: Provides methods for the addition of an external entity to the mapping
- *`PX.Commerce.Core.IMappedEntityLocal<LocalType>`*: Provides a method for the addition of an internal entity to the mapping

The *`MappedEntity<ExternType, LocalType>`* class provides the default implementation of these interfaces. The class also provides the default constructors from which you can derive the constructors of the mapping classes. These constructors use the following as input parameters:

- A `string` identifier of the mapped entities, which is exactly two characters long
- A `string` identifier of the connector, which is defined in the `ConnectorType` property of the connector class

Example

The following code shows an example of the implementation of a mapping class.

> You can see this code on *GitHub*.

```
using System;
using PX.Commerce.Core;
using PX.Commerce.Core.API;

namespace WooCommerceTest
{
    public class MappedCustomer : MappedEntity<CustomerData, Customer>
    {
        public const string TYPE = BCEntitiesAttribute.Customer;

        public MappedCustomer()
            : base(WooCommerceConnector.TYPE, TYPE)
        { }
        public MappedCustomer(Customer entity, Guid? id, DateTime? timestamp)
            : base(WooCommerceConnector.TYPE, TYPE, entity, id, timestamp) { }
        public MappedCustomer(CustomerData entity, string id,
            DateTime? timestamp)
            : base(WooCommerceConnector.TYPE, TYPE, entity,
            id, id,timestamp) { }
    }
}
```

Connector Implementation: Bucket Classes

For the synchronization of data between an external system and Acumatica ERP, for each entity that you need to synchronize, you need to define a bucket class that defines the entities to be synchronized before and after the synchronization of this entity.

Base Class and Interface

You define this bucket in the bucket class, which derives from the *`PX.Commerce.Core.IEntityBucket`* interface and, optionally, the *`PX.Commerce.Core.EntityBucketBase`* base class. The `EntityBucketBase` class is a supplementary class that provides the default implementation of the `PreProcessors` and `PostProcessors` properties of the `IEntityBucket` interface.

In the bucket class, you need to define the entity that is being synchronized in the `Primary` property of the bucket class. In the `Entities` property of the bucket class, you specify all entities that are being synchronized in this bucket. The `PreProcessors` property contains the list of entities that should be synchronized before the entity is, while the `PostProcessors` property specifies the list of entities that should be synchronized after the entity is.

Example

Suppose that after synchronizing address entities between an external system and Acumatica ERP, you need to synchronize customer entities. The following code shows an example of the bucket class implementation for the address and customer entities.

> You can see this code on *GitHub*.

```
using PX.Commerce.BigCommerce;
using PX.Commerce.Core;

namespace WooCommerceTest
{
    public class WooLocationEntityBucket : EntityBucketBase, IEntityBucket
    {
        public IMappedEntity Primary => Address;
        public IMappedEntity[] Entities =>
          new IMappedEntity[] { Address, Customer };

        public override IMappedEntity[] PostProcessors =>
          new IMappedEntity[] { Customer };

        public MappedLocation Address = null!;
        public MappedCustomer Customer = null!;
    }
}
```

Connector Implementation: DAC for the Connector's Configuration Settings

You need to create a DAC with the configuration settings that are used by the connector. For this DAC, you also need to add a database table and include it in the customization project.

Creation of the Database Table

In the database of the instance that you are using for the development of the connector, you create a database table with the settings used by the connector. An example of the SQL script for this table is shown below.

```
GO
IF NOT EXISTS (SELECT * FROM SYSOBJECTS
  WHERE NAME='BCBINDINGWOOCOMMERCE' AND XTYPE='U')
BEGIN
  CREATE TABLE [dbo].[BCBindingWooCommerce](
 [CompanyID] [int] NOT NULL,
 [BindingID] [int] NOT NULL,
 [StoreBaseUrl] [nvarchar](100) NULL,
 [StoreXAuthToken] [nvarchar](max) NULL,
 [StoreXAuthClient] [nvarchar](max) NULL,
 [StoreAdminUrl] [nvarchar](200) NULL,
 [WooCommerceStoreTimeZone] [nvarchar](100) NULL,
 [WooCommerceDefaultCurrency] [nvarchar](12) NULL,
 [tstamp] [timestamp] NOT NULL,
 CONSTRAINT [PK_BCBindingWooCommerce] PRIMARY KEY CLUSTERED
(
 [CompanyID] ASC,
 [BindingID] ASC
)WITH (PAD_INDEX = OFF, STATISTICS_NORECOMPUTE = OFF, IGNORE_DUP_KEY = OFF,
ALLOW_ROW_LOCKS = ON, ALLOW_PAGE_LOCKS = ON) ON [PRIMARY]
) ON [PRIMARY] TEXTIMAGE_ON [PRIMARY]

ALTER TABLE [dbo].[BCBindingWooCommerce] ADD  DEFAULT ((0)) FOR [CompanyID]
END
```

You then add the table to the customization project, as described in *To Add a Custom Table to a Project*.

Creation of the DAC

In the Visual Studio project of the extension library, you create the DAC that stores the connector settings, as shown in the following example.

- You can generate the DAC from the table in the database by using the Customization Project Editor (as described in *To Create a New DAC*) and move it to the extension library (as described in *To Move a Code Item to the Extension Library*). Alternatively, you can create the DAC directly in the Visual Studio project.
- You can see this code on *GitHub*.

```
using PX.Commerce.Core;
using PX.Data;
using PX.Data.ReferentialIntegrity.Attributes;

namespace WooCommerceTest
{
    [PXCacheName("WooCommerce Settings")]
    public class BCBindingWooCommerce : PXBqlTable, IBqlTable
    {
        public class PK : PrimaryKeyOf<BCBindingWooCommerce>.
            By<BCBindingWooCommerce.bindingID>
        {
            public static BCBindingWooCommerce Find(
```

```
            PXGraph graph, int? binding) => FindBy(graph, binding);
        }

        #region BindingID
        [PXDBInt(IsKey = true)]
        [PXDBDefault(typeof(BCBinding.bindingID))]
        [PXUIField(DisplayName = "Store", Visible = false)]
        [PXParent(typeof(Select<BCBinding, Where<BCBinding.bindingID,
            Equal<Current<BCBindingWooCommerce.bindingID>>>>))]
        public int? BindingID { get; set; }
        public abstract class bindingID :
            PX.Data.BQL.BqlInt.Field<bindingID> { }
        #endregion

        //Connection
        #region StoreBaseUrl
        [PXDBString(50, IsUnicode = true, InputMask = "")]
        [PXUIField(DisplayName = "API Path")]
        [PXDefault()]
        public virtual string? StoreBaseUrl { get; set; }
        public abstract class storeBaseUrl :
            PX.Data.BQL.BqlString.Field<storeBaseUrl> { }
        #endregion
        #region StoreXAuthClient
        [PXRSACryptString(IsUnicode = true, InputMask = "")]
        [PXUIField(DisplayName = "Consumer Key")]
        [PXDefault()]
        public virtual string? StoreXAuthClient { get; set; }
        public abstract class storeXAuthClient :
            PX.Data.BQL.BqlString.Field<storeXAuthClient> { }
        #endregion
        #region StoreXAuthToken
        [PXRSACryptString(IsUnicode = true, InputMask = "")]
        [PXUIField(DisplayName = "Consumer Secret")]
        [PXDefault()]
        public virtual string? StoreXAuthToken { get; set; }
        public abstract class storeXAuthToken :
            PX.Data.BQL.BqlString.Field<storeXAuthToken> { }
        #endregion

        #region WooCommerceDefaultCurrency
        [PXDBString(12, IsUnicode = true)]
        [PXUIField(DisplayName = "Default Currency", IsReadOnly = true)]
        public virtual string? WooCommerceDefaultCurrency { get; set; }
        public abstract class wooCommerceDefaultCurrency :
            PX.Data.BQL.BqlString.Field<wooCommerceDefaultCurrency> { }
        #endregion

        #region WooCommerceStoreTimeZone
        [PXDBString(100, IsUnicode = true)]
        [PXUIField(DisplayName = "Store Time Zone", IsReadOnly = true)]
        public virtual string? WooCommerceStoreTimeZone { get; set; }
        public abstract class wooCommerceStoreTimeZone :
            PX.Data.BQL.BqlString.Field<wooCommerceStoreTimeZone> { }
        #endregion

        #region StoreAdminURL
```

```
        [PXDBString(100, IsUnicode = true, InputMask = "")]
        [PXUIField(DisplayName = "Store Admin Path")]
        [PXDefault()]
        public virtual string? StoreAdminUrl { get; set; }
        public abstract class storeAdminUrl :
            PX.Data.BQL.BqlString.Field<storeAdminUrl> { }
        #endregion

    }
}
```

Connector Implementation: API Client

You need to implement an API client of the external system. The built-in connectors in Acumatica ERP use REST API clients or GraphQL clients. The details of the implementation of an API client depend on the external system. In this topic, you can find descriptions of particular approaches that can be used for implementation.

> The external system determines which classes you need to implement and how the connector communicates with the external system.

Processing of API Rate Limits

You can implement the following approach for handling the *429 Too Many Requests* response code: If this response code is received, the connector checks the headers of the response to find the recommended delay for the request, and retries the request after this delay.

To use this approach, you need to derive the REST API client implementation from the `PX.Commerce.Core.REST.RetryCapableRESTClientBase` abstract class and override its methods. The `RetryCapableRESTClientBase` class uses the `Polly.Core.dll` library.

The following code shows a sample fragment of a REST API client implementation.

> You can see this code on *GitHub*.

```
using PX.Commerce.BigCommerce;
using PX.Commerce.Core;
using PX.Data;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using PX.Commerce.Core.REST;
using Polly;
using PX.Commerce.BigCommerce.API.REST;
using System.Net.Http;

namespace WooCommerceTest
{
    public abstract class WooRestClientBase :
        RetryCapableRESTClientBase<WooRestClientBase.HttpCallContext>
    {
        protected WooRestClientBase(int attempts) : base(attempts)
```

```
        {
        }

        protected override ValueTask<bool> IsFailure(Outcome<HttpCallContext> result,
            int attemptNumber, int attempts) =>
            new((result.Exception is not null
                || (result.Result?.Response is not null &&
 (int)result.Result.Response.StatusCode == 429))
                && attempts >= attemptNumber);

        protected override ValueTask HandleError(HttpCallContext ctx,
            int attemptNumber, int attempts)
        {
            if (attempts == attemptNumber)
                throw new PXException(BCMessages.RetryLimitIsExceeded);

            return new ValueTask();
        }

        protected override ValueTask<TimeSpan?> GetDelay(HttpCallContext context,
            int attemptNumber)
        {
            if (context is null)
                return new(TimeSpan.FromSeconds(attemptNumber + 1));

            var response = context.Response;
            if ((int)response.StatusCode == 429)
            {
                if ((response.Headers.TryGetValues(
                    BigCommerceConstants.Headers.RateLimitResetMs, out var values)
                    || response.Headers.TryGetValues(
                        BigCommerceConstants.Headers.RateLimitWindowMs, out values))
                    && int.TryParse(values.FirstOrDefault(), out var delay))
                    return new ValueTask<TimeSpan?>(TimeSpan.FromMilliseconds(delay));

                return new ValueTask<TimeSpan?>(TimeSpan.FromSeconds(attemptNumber + 1));
            }

            return new ValueTask<TimeSpan?>((TimeSpan?)null);
        }

        public class HttpCallContext(IBCRestResponse Response, HttpRequestMessage Request,
 string Body)
        {
            public IBCRestResponse Response { get; } = Response;
            public HttpRequestMessage Request { get; } = Request;
            public string Body { get; } = Body;
        }
    }
}
```

Connector Implementation: Processor Classes

For each pair of entities that you want to synchronize between an external system and Acumatica ERP, you need to create a processor class.

Processor classes perform the following functions:

- Retrieval of the Acumatica ERP records and external records
- Providing of the default mapping logic
- Providing of the export and import logic

Base Classes and Interface

A processor class implements the *PX.Commerce.Core.IProcessor* interface and derives from one of the following base abstract classes:

- *PX.Commerce.Core.BCProcessorSingleBase<TGraph, TEntityBucket, TPrimaryMapped>*, which is a standard base class for processors that synchronizes records one by one.
- PX.Commerce.Core.BCProcessorBulkBase<TGraph, TEntityBucket, TPrimaryMapped>, which can mass-process records for the entities that require higher performance during synchronization. For example, this base class is used for the processor classes for the product availability and price list entities in the BigCommerce connector.

Both of these classes are graphs. Therefore, the processor class is a graph as well. In the processor base class, you pass as the type parameters of the class the graph itself, the bucket class, and the mapping class for the pair of entities that you want to synchronize.

Attributes of the Processor Class

You assign the following attributes to the processor class:

- *BCProcessor*, which specifies the basic functions of the processor, such as which directions of synchronization are supported by the processor
- Optional: *BCProcessorRealtime*, which specifies whether the processor works with push notifications and webhooks

Methods of the Processor Class

In the processor class, you implement the methods that perform the following functions:

- The fetching of the records that have been changed in Acumatica ERP or the external system
- The checking and merging of duplicated records
- The handling of real-time processing
- Other supplementary functions

For details about the methods, see *PX.Commerce.Core.IProcessor*, *PX.Commerce.Core.BCProcessorSingleBase<TGraph, TEntityBucket, TPrimaryMapped>*, and PX.Commerce.Core.BCProcessorBulkBase<TGraph, TEntityBucket, TPrimaryMapped>.

Example

The following code shows an example of the processor implementation.

You can see this code on *GitHub*.

```
using PX.Api.ContractBased.Models;
using PX.Commerce.Core;
using PX.Commerce.Core.API;
using PX.Commerce.Objects;
using PX.Data;
using PX.Objects.AR;
using RestSharp;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Threading;
using System.Threading.Tasks;

namespace WooCommerceTest
{
    [BCProcessor(typeof(WooCommerceConnector), BCEntitiesAttribute.Customer,
        BCCaptions.Customer, 20,
        IsInternal = false,
        Direction = SyncDirection.Bidirect,
        PrimaryDirection = SyncDirection.Import,
        PrimarySystem = PrimarySystem.Extern,
        PrimaryGraph = typeof(PX.Objects.AR.CustomerMaint),
        ExternTypes = new Type[] { typeof(CustomerData) },
        LocalTypes = new Type[] { typeof(PX.Commerce.Core.API.Customer) },
        AcumaticaPrimaryType = typeof(PX.Objects.AR.Customer),
        AcumaticaPrimarySelect = typeof(PX.Objects.AR.Customer.acctCD),
        URL = "user-edit.php?user_id={0}"
    )]
    [BCProcessorRealtime(
        PushSupported = true,
        PushSources = new String[] { "BC-PUSH-Customers" },
        PushDestination = WCCaptions.PushDestination,
        HookSupported = true,
        WebHookType = typeof(CustomerData),
        WebHooks = new String[]
        {
            "customer.created",
            "customer.updated",
            "customer.deleted"
        }
    )]
    public class WooCustomerProcessor :
        BCProcessorSingleBase<WooCustomerProcessor,
            WooCustomerEntityBucket, MappedCustomer>, IProcessor
    {
        public RestClient client = null!;
        protected CustomerDataProvider customerDataProvider = null!;

        //protected List<Country> countries;
        public CommerceHelper helper =
            PXGraph.CreateInstance<CommerceHelper>();

        #region Import
```

```
        public override async Task MapBucketImport(
            WooCustomerEntityBucket bucket, IMappedEntity existing,
            CancellationToken cancellationToken = default)
        {
            MappedCustomer customerObj = bucket.Customer;

            PX.Commerce.Core.API.Customer customerImpl = customerObj.Local =
                new PX.Commerce.Core.API.Customer();
            customerImpl.Custom = GetCustomFieldsForImport();

            customerImpl.CustomerName =
                GetBillingCustomerName(customerObj.Extern).ValueField();
            customerImpl.AccountRef = APIHelper.ReferenceMake(
                customerObj.Extern.Id, GetBinding().BindingName).ValueField();
            customerImpl.CustomerClass = customerObj.LocalID == null ||
                existing?.Local == null ?
                GetBindingExt<BCBindingExt>().CustomerClassID?.ValueField() :
                    null;

            //Primary Contact
            customerImpl.PrimaryContact =
                GetPrimaryContact(customerObj.Extern);

            customerImpl.MainContact = SetBillingContact(customerObj.Extern);

            customerImpl.ShippingAddressOverride = true.ValueField();
            customerImpl.ShippingContactOverride = true.ValueField();
            customerImpl.ShippingContact =
                SetShippingContact(customerObj.Extern);

            BCBindingExt bindingExt = GetBindingExt<BCBindingExt>();
            if (bindingExt.CustomerClassID != null)
            {
                CustomerClass customerClass = PXSelect<CustomerClass,
                    Where<CustomerClass.customerClassID,
                        Equal<Required<CustomerClass.customerClassID>>>>.
                            Select(this, bindingExt.CustomerClassID);
                if (customerClass != null)
                {
                    customerClass.ShipVia.ValueField();

                }
            }

        }

        public virtual Contact GetPrimaryContact(CustomerData customer)
        {
            var contact = new Contact();
            contact.FirstName = !string.IsNullOrWhiteSpace(customer.FirstName)
                && string.IsNullOrWhiteSpace(customer.LastName) ?
                string.Empty.ValueField() : customer.FirstName.ValueField();
            contact.LastName = !string.IsNullOrWhiteSpace(customer.LastName) ?
                customer.LastName.ValueField() :
                !string.IsNullOrWhiteSpace(customer.FirstName) &&
                string.IsNullOrWhiteSpace(customer.LastName) ?
                customer.FirstName.ValueField() :
```

```
            customer.Username.ValueField();
        contact.Email = customer.Email.ValueField();

        return contact;
    }

    public virtual Contact SetBillingContact(CustomerData customerObj)
    {
        Contact contactImpl = new Contact();
        contactImpl.DisplayName =
            GetBillingCustomerName(customerObj).ValueField();
        contactImpl.Attention =
        $"{customerObj.Billing?.FirstName} {customerObj.Billing?.LastName}"
        .ValueField(); ;
        contactImpl.FullName = GetBillingCustomerName(customerObj)
        .ValueField();
        contactImpl.FirstName = customerObj.Billing?.FirstName
        .ValueField();
        contactImpl.LastName = customerObj.Billing?.LastName?.ValueField();
        contactImpl.Email = customerObj.Billing?.Email?.ValueField();
        contactImpl.Address = MapAddress(customerObj.Billing);
        contactImpl.Active = true.ValueField();
        contactImpl.Phone1 = customerObj.Billing?.Phone.ValueField();
        contactImpl.Phone1Type = PhoneTypes.Business1.ValueField();

        return contactImpl;
    }

    public virtual Contact SetShippingContact(CustomerData customerObj)
    {
        Contact contactImpl = new Contact();
        contactImpl.DisplayName = GetShippingCustomerName(customerObj)
        .ValueField();
        contactImpl.Attention =
        $"{customerObj.Shipping?.FirstName} {customerObj.Shipping?.LastName}"
        .ValueField();
        contactImpl.FullName = GetShippingCustomerName(customerObj)
        .ValueField();
        contactImpl.FirstName = customerObj.Shipping?.FirstName
        .ValueField();
        contactImpl.LastName = customerObj.Shipping.LastName.ValueField();
        contactImpl.Address = MapAddress(customerObj.Shipping);
        contactImpl.Active = true.ValueField();

        return contactImpl;
    }

    public virtual string GetBillingCustomerName(CustomerData data)
    {
        return !string.IsNullOrWhiteSpace(data.Billing.Company)
            ? data.Billing.Company
            : string.IsNullOrWhiteSpace(
            $"{data.Billing?.FirstName} {data.Billing?.LastName}")
            ? data.Username :
            $"{data.Billing?.FirstName} {data.Billing?.LastName}";
    }
```

```
        public virtual string GetShippingCustomerName(CustomerData data)
        {
            return !string.IsNullOrWhiteSpace(data.Shipping?.Company)
                ? data.Shipping.Company
                : $"{data.Shipping?.FirstName} {data.Shipping?.LastName}";
        }

        public virtual Address MapAddress(CustomerAddressData addressObj)
        {
            var address = new Address();
            address.AddressLine1 = addressObj.Address1.ValueField();
            address.AddressLine2 = addressObj.Address2.ValueField();
            address.City = addressObj.City.ValueField();
            address.PostalCode = addressObj.PostalCode.ValueField();
            if (!string.IsNullOrWhiteSpace(addressObj.Country))
            {
                var selectedCountry = addressObj.Country;
                if (selectedCountry != null)
                {
                    address.Country = addressObj.Country.ValueField();
                    if (!string.IsNullOrWhiteSpace(addressObj.State))
                    {
                        address.State = addressObj.State.ValueField();
                    }
                }
            }

            return address;
        }

        public override async Task<PullSimilarResult<MappedCustomer>>
            PullSimilar(IExternEntity entity,
            CancellationToken cancellationToken = default)
        {
            var uniqueField = string.IsNullOrWhiteSpace(
              ((CustomerData)entity)?.Billing.Email) ?
              ((CustomerData)entity)?.Email :
              ((CustomerData)entity)?.Billing.Email;
            if (uniqueField == null) return null;

            List<MappedCustomer> result = new List<MappedCustomer>();
            foreach (PX.Objects.AR.Customer item in
                helper.CustomerByEmail.Select(uniqueField))
            {
                PX.Commerce.Core.API.Customer data =
                    new PX.Commerce.Core.API.Customer() {
                      SyncID = item.NoteID,
                      SyncTime = item.LastModifiedDateTime };
                result.Add(
                    new MappedCustomer(data, data.SyncID, data.SyncTime));
            }

            if (result == null || result?.Count == 0) return null;

            return new PullSimilarResult<MappedCustomer>() {
                UniqueField = uniqueField, Entities = result };
        }
```

```
public override async Task<PullSimilarResult<MappedCustomer>>
    PullSimilar(ILocalEntity entity,
    CancellationToken cancellationToken = default)
{
    var uniqueField = ((PX.Commerce.Core.API.Customer)entity)?.
        MainContact?.Email?.Value;
    if (uniqueField == null) return null;
    IEnumerable<CustomerData> datas = customerDataProvider.GetAll();
    if (datas == null) return null;

    return new PullSimilarResult<MappedCustomer>() {
        UniqueField = uniqueField,
        Entities = datas.Select(data => new MappedCustomer(
        data, data.Id.ToString(), data.DateModified.ToDate())) };
}

public override async Task SaveBucketImport(
    WooCustomerEntityBucket bucket, IMappedEntity existing,
    string operation, CancellationToken cancellationToken = default)
{
    MappedCustomer obj = bucket.Customer;

    PX.Commerce.Core.API.Customer impl = cbapi.Put(obj.Local,
        obj.LocalID);

    obj.AddLocal(impl, impl.SyncID, impl.SyncTime);
    UpdateStatus(obj, operation);
}
#endregion

#region Export

public override async Task FetchBucketsForExport(
    DateTime? minDateTime, DateTime? maxDateTime,
    PXFilterRow[] filters, CancellationToken cancel = default)
{
    IEnumerable<PX.Commerce.Core.API.Customer> impls =
        cbapi.GetAll<PX.Commerce.Core.API.Customer>(
        new PX.Commerce.Core.API.Customer {
            CustomerID = new StringReturn() },
        minDateTime, maxDateTime, filters);

    int countNum = 0;
    List<IMappedEntity> mappedList = new List<IMappedEntity>();
    foreach (PX.Commerce.Core.API.Customer impl in impls)
    {
        IMappedEntity obj = new MappedCustomer(impl, impl.SyncID,
            impl.SyncTime);

        mappedList.Add(obj);
        countNum++;
        if (countNum % BatchFetchCount == 0)
        {
            ProcessMappedListForExport(mappedList);
        }
    }
```

```
            if (mappedList.Any())
            {
                ProcessMappedListForExport(mappedList);
            }
        }

        public override async Task MapBucketExport(
            WooCustomerEntityBucket bucket, IMappedEntity existing,
            CancellationToken cancel = default)
        {
            MappedCustomer customerObj = bucket.Customer;

            PX.Commerce.Core.API.Customer customerImpl = customerObj.Local;
            Contact contactImpl = customerImpl.MainContact;
            Contact primaryContact = customerImpl.PrimaryContact;
            Address addressImpl = contactImpl.Address;
            CustomerData customerData = customerObj.Extern =
                new CustomerData();

            //Customer
            customerData.Id = customerObj.ExternID.ToInt();

            //Contact
            // Use the primary contact's first name and last name
            // as the customer's first name and last name
            // if the primary contact exists.
            // Otherwise, use the customer name.
            customerData.FirstName = primaryContact?.FirstName?.Value ??
                primaryContact?.LastName?.Value ??
                customerImpl.CustomerName?.Value.FieldsSplit(
                0, customerImpl.CustomerName?.Value);
            customerData.LastName = primaryContact?.LastName?.Value ??
                primaryContact?.FirstName?.Value ??
                customerImpl.CustomerName?.Value.FieldsSplit(
                1, customerImpl.CustomerName?.Value);
            customerData.Email = contactImpl.Email?.Value;

            if (!string.IsNullOrEmpty(customerImpl.PriceClassID?.Value))
            {
                BCSyncStatus status = PXSelectJoin<BCSyncStatus,
                    LeftJoin<PX.Objects.AR.ARPriceClass,
                        On<BCSyncStatus.localID,
                            Equal<PX.Objects.AR.ARPriceClass.noteID>>>,
                    Where<BCSyncStatus.connectorType,
                        Equal<Current<BCEntity.connectorType>>,
                        And<BCSyncStatus.bindingID,
                            Equal<Current<BCEntity.bindingID>>,
                        And<BCSyncStatus.entityType,
                            Equal<Required<BCEntity.entityType>>,
                        And<PX.Objects.AR.ARPriceClass.priceClassID,
                            Equal<Required<
                                PX.Objects.AR.ARPriceClass.priceClassID>>>>>>>.
                    Select(this, BCEntitiesAttribute.CustomerPriceClass,
                        customerImpl.PriceClassID?.Value);
                if (GetEntity(BCEntitiesAttribute.CustomerPriceClass)?.
                    IsActive == true)
                {
```

```
                if (status?.ExternID == null) throw new PXException(
                    BCMessages.PriceClassNotSyncronizedForItem,
                    customerImpl.PriceClassID?.Value);
            }
        }
    }

    public override async Task SaveBucketExport(
        WooCustomerEntityBucket bucket, IMappedEntity existing,
        string operation, CancellationToken cancel = default)
    {
        MappedCustomer obj = bucket.Customer;

        //Customer
        CustomerData customerData = null;
        obj.AddExtern(customerData, customerData.Id?.ToString(),
            customerData.LastName, customerData.DateCreatedUT.ToDate());

        UpdateStatus(obj, operation);

        #region Update ExternalRef
        string externalRef = APIHelper.ReferenceMake(
            customerData.Id?.ToString(), GetBinding().BindingName);

        string[] keys = obj.Local?.AccountRef?.Value?.Split(';');
        if (keys?.Contains(externalRef) != true)
        {
            if (!string.IsNullOrEmpty(obj.Local?.AccountRef?.Value))
                externalRef = new object[] { obj.Local?.AccountRef?.Value,
                    externalRef }.KeyCombine();

            if (externalRef.Length < 50 && obj.Local.SyncID != null)
                PXDatabase.Update<PX.Objects.CR.BAccount>(
                    new PXDataFieldAssign(
                    typeof(PX.Objects.CR.BAccount.acctReferenceNbr).Name,
                    PXDbType.NVarChar, externalRef),
                    new PXDataFieldRestrict(
                    typeof(PX.Objects.CR.BAccount.noteID).Name,
                    PXDbType.UniqueIdentifier, obj.Local.NoteID?.Value)
                );
        }
        #endregion

    }
    #endregion

    #region Pull
    public override async Task<MappedCustomer> PullEntity(Guid? localID,
        Dictionary<string, object> externalInfo,
            CancellationToken cancellationToken = default)
    {
        PX.Commerce.Core.API.Customer impl =
            cbapi.GetByID<PX.Commerce.Core.API.Customer>(localID);
        if (impl == null) return null;

        MappedCustomer obj = new MappedCustomer(impl, impl.SyncID,
            impl.SyncTime);
```

```
        return obj;
    }
    public override async Task<MappedCustomer> PullEntity(string externID,
        string externalInfo, CancellationToken cancellationToken = default)
    {
        client = WooCommerceConnector.GetRestClient(
            GetBindingExt<BCBindingWooCommerce>());
        customerDataProvider = new CustomerDataProvider(client);
        CustomerData data = customerDataProvider.GetAll().FirstOrDefault();
        if (data == null) return null;

        MappedCustomer obj = new MappedCustomer(data, data.Id?.ToString(),
            data.DateModified.ToDate());
        return obj;
    }

    public override async Task FetchBucketsForImport(DateTime? minDateTime,
        DateTime? maxDateTime, PXFilterRow[] filters,
        CancellationToken cancellationToken = default)
    {
        IEnumerable<CustomerData> customers = customerDataProvider.GetAll();

        foreach (CustomerData data in customers)
        {
            WooCustomerEntityBucket bucket = CreateBucket();

            MappedCustomer mapped = bucket.Customer = bucket.Customer.Set(
                data, data.Id.ToString(), data.LastName,
                data.DateModified.ToDate());
            EnsureStatus(mapped, SyncDirection.Import);
        }
    }

    public override async Task<EntityStatus> GetBucketForImport(
        WooCustomerEntityBucket bucket, BCSyncStatus syncstatus,
        CancellationToken cancel = default)
    {
        CustomerData data = client.Get<CustomerData>(
            new RestRequest(string.Format("/customers/{0}",
            syncstatus.ExternID)));

        MappedCustomer obj = bucket.Customer = bucket.Customer.Set(data,
            data.Id.ToString(), data.LastName, data.DateModified.ToDate());
        EntityStatus status = EnsureStatus(obj, SyncDirection.Import);

        return status;
    }

    public override async Task<EntityStatus> GetBucketForExport(
        WooCustomerEntityBucket bucket, BCSyncStatus bcstatus,
        CancellationToken cancellationToken = default)
    {
        PX.Commerce.Core.API.Customer impl = cbapi.GetByID<
            PX.Commerce.Core.API.Customer>(bcstatus.LocalID,
            GetCustomFieldsForExport());
        if (impl == null) return EntityStatus.None;
```

```
            bucket.Customer = bucket.Customer.Set(impl, impl.SyncID,
                impl.SyncTime);
            EntityStatus status = EnsureStatus(bucket.Customer,
                SyncDirection.Export);

            return status;
        }
        #endregion
    }
    public static class PhoneTypes
    {
        public static string Business1 = "B1";
    }
}
```

Connector Implementation: Connector Class

The connector class is the main class of a connector for an external system.

The connector class performs the following functions:

- Provides the settings for connection with the external system
- Implements navigation to external records
- Performs the synchronization of records of the external system and Acumatica ERP
- Implements real-time subscription and processing

Base Class and Interface

The connector class derives from the *`PX.Commerce.Core.BCConnectorBase<TGraph>`* base abstract class. The `BCConnectorBase<TGraph>` class implements the *`PX.Commerce.Core.IConnector`* interface. Because the `BCConnectorBase<TGraph>` class is a graph, the connector class is a graph as well.

Properties and Methods of the Connector Class

In the `ConnectorType` property of the connector class, you define a `string` identifier of the connector type; this identifier can be no more than three characters long. In the `ConnectorName` property of this class, you define the name of the connector, which is displayed on Acumatica ERP forms. For example, the value is displayed in the **Connector** box on the *Entities* (BC202000) form.

For details about the methods of the connector class, see *`PX.Commerce.Core.IConnector`*.

Example

The following code shows an example of the implementation of the connector class.

> You can see this code on *GitHub*.

```
using Newtonsoft.Json;
using PX.Async;
using PX.Commerce.BigCommerce;
using PX.Commerce.BigCommerce.API.REST;
```

```
using PX.Commerce.Core;
using PX.Commerce.Core.REST;
using PX.Common;
using PX.Data;
using PX.Data.BQL;
using RestSharp;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text.RegularExpressions;
using System.Threading;
using System.Threading.Tasks;

namespace WooCommerceTest
{
    public class WooCommerceConnector : BCConnectorBase<WooCommerceConnector>
    {
        public const string TYPE = "WOO";
        public const string NAME = "WooCommerce";

        public class WCConnectorType : BqlString.Constant<WCConnectorType>
        {
            public WCConnectorType() : base(TYPE) { }
        }

        public override string ConnectorType { get => TYPE; }
        public override string ConnectorName { get => NAME; }

        public override void NavigateExtern(ISyncStatus status,
            ISyncDetail detail)
        {
            if (status?.ExternID == null) return;

            EntityInfo info = GetEntities().FirstOrDefault(e =>
                e.EntityType == status.EntityType);
            BCBindingWooCommerce bCBindingBigCommerce =
                BCBindingWooCommerce.PK.Find(this, status.BindingID);

            if (string.IsNullOrEmpty(bCBindingBigCommerce?.StoreAdminUrl) ||
                string.IsNullOrEmpty(info.URL)) return;

            string[] parts = status.ExternID.Split(new char[] { ';' });
            string url = string.Format(info.URL, parts.Length > 2 ?
                parts.Take(2).ToArray() : parts);
            string redirectUrl =
                bCBindingBigCommerce.StoreAdminUrl.TrimEnd('/') + "/" + url;

            throw new PXRedirectToUrlException(redirectUrl,
                PXBaseRedirectException.WindowMode.New, string.Empty);
        }

        public override async Task<ConnectorOperationResult> Process(
            ConnectorOperation operation, int?[] syncIDs,
            CancellationToken cancellationToken = default)
        {
            EntityInfo info = GetEntities().FirstOrDefault(e =>
                e.EntityType == operation.EntityType);
```

```
            using (IProcessor graph = (IProcessor)CreateInstance(
                info.ProcessorType))
            {
                await graph.Initialise(this, operation);
                return await graph.Process(syncIDs, cancellationToken);
            }
        }

        public override async Task<DateTime> GetSyncTime(ConnectorOperation operation)
        {
            BCBindingWooCommerce binding = BCBindingWooCommerce.PK.Find(this,
                operation.Binding);
            //Acumatica Time
            PXDatabase.SelectDate(out DateTime dtLocal, out DateTime dtUtc);
            dtLocal = PX.Common.PXTimeZoneInfo.ConvertTimeFromUtc(dtUtc,
                PX.Common.LocaleInfo.GetTimeZone());

            return dtLocal;
        }

        public override async Task ProcessHook(
            IEnumerable<BCExternQueueMessage> messages,
                CancellationToken cancellationToken = default)
        {
            throw new NotImplementedException();
        }

        public static RestClient GetRestClient(BCBindingWooCommerce binding)
        {
            Dictionary<string, string> authHeaders = new()
            {
                { BigCommerceConstants.Headers.AuthToken, binding.StoreXAuthToken },
                { BigCommerceConstants.Headers.AuthClient, binding.StoreXAuthClient }
            };
            return CreateClient(binding.StoreBaseUrl, authHeaders);
        }

        public static RestClient CreateClient(string baseUri, Dictionary<string, string>
authHeaders)
        {
            RestOptions options = new RestOptions
            {
                BaseUri = baseUri,
                AuthHeaders = authHeaders
            };
            JsonSerializerSettings serializer = new JsonSerializerSettings
            {
                MissingMemberHandling = MissingMemberHandling.Ignore,
                NullValueHandling = NullValueHandling.Ignore,
                DefaultValueHandling = DefaultValueHandling.Include,
                DateFormatHandling = DateFormatHandling.IsoDateFormat,
                DateTimeZoneHandling = DateTimeZoneHandling.Unspecified,
                ContractResolver = new GetOnlyContractResolver()
            };

            RestClient client = new RestClient();
```

```
            return client;
        }

        public List<Tuple<string, string, string>> GetExternalFields(
            string type, int? binding, string entity)
        {
            List<Tuple<string, string, string>> fieldsList =
                new List<Tuple<string, string, string>>();
            if (entity != BCEntitiesAttribute.Customer && entity !=
                BCEntitiesAttribute.Address) return fieldsList;

            return fieldsList;
        }

        public override Task StartWebHook(string baseUrl, BCWebHook hook,
            CancellationToken cancellationToken = default)
        {
            throw new NotImplementedException();
        }

        public override Task StopWebHook(string baseUrl, BCWebHook hook,
            CancellationToken cancellationToken = default)
        {
            throw new NotImplementedException();
        }
    }
}
```

Connector Implementation: Connector's Factory Class

A connector's factory class initializes a connector that has the specified type and name.

Base Class and Interface

The connector's factory class implements the *`PX.Commerce.Core.IConnectorFactory`* interface and, optionally, derives from the *`PX.Commerce.Core.BaseConnectorFactory<GraphType>`* base abstract class, which provides the default implementation of particular methods of the interface.

Members of the Class

In the connector's factory class, you specify the type of the connector in the `Type` property and the name of the connector in the `Description` property. These are the type and name that are defined in the `ConnectorType` and `ConnectorName` properties of the connector class.

You also define the condition when the connector is available in Acumatica ERP in the `Enabled` property. For example, the connector may be available if a particular feature is enabled on the *Enable/Disable Features* (CS100000) form.

If your connector's factory class derives from the `BaseConnectorFactory<GraphType>`, you need to implement only the following items: a constructor of the factory class, and the `GenerateExternID()` and `GenerateLocalID()` methods.

Example

The following code shows an example of the implementation of a connector's factory class.

> You can see this code on *GitHub*.

```
using PX.Commerce.Core;
using System;

namespace WooCommerceTest
{
    public class WooCommerceConnectorFactory :
        BaseConnectorFactory<WooCommerceConnector>, IConnectorFactory
    {
        public override string Description => WooCommerceConnector.NAME;
        public override bool Enabled => true;

        public override string Type => WooCommerceConnector.TYPE;

        public WooCommerceConnectorFactory(ProcessorFactory factory)
            : base(factory)
        {
        }

        public virtual Guid? GenerateExternID(BCExternNotification message)
        {
            throw new NotImplementedException();
        }
    }
}
```

Connector Implementation: How the Connector Is Detected in the Application

The **Connector** box of the configuration form of an ecommerce store contains the name of the connector if both of the following conditions are met:

- The library that contains the connector class is placed in the `Bin` folder of an instance of Acumatica ERP.
- The ASPX file of the configuration form of the connector is available in the `Pages` folder of the application.

> The description in this topic assumes that the connector uses a standard configuration form, which is similar to the *BigCommerce Stores* (BC201000) and *Shopify Stores* (BC201010) forms.

The data that is displayed in the Summary area of the configuration form is obtained from the `Bindings` predefined data view of the `BCStoreMaint` graph, which is a base class of the graph through which the configuration form is managed. The `Bindings` data view retrieves data from the `BCBinding` predefined database table. This table includes the `ConnectorType` column, which stores the type of the connector.

The system obtains the type and name of the connector from the implementation of the `Type` and `Description` properties of the *`IConnectorFactory`* interface. The `ConnectorType` field of the `BCBinding` DAC, which corresponds to the *`BCBinding`* table, has the `BCConnectors` attribute assigned to it. The `BCConnectors` attribute, which derives from the `PXStringList` attribute, matches the type of the connector to its name. The custom `CacheAttached<BCBinding.connectorType>` event handler sets the default value of the

`ConnectorType` field to the type of the connector. The **Connector** box displays the connector name, which corresponds to this default value.

When a user specifies the settings of a commerce store in the Summary area of the configuration form, the system saves these settings in the `BCBinding` database table.

The following diagram illustrates the relations of the classes of the connector and the ASPX page.

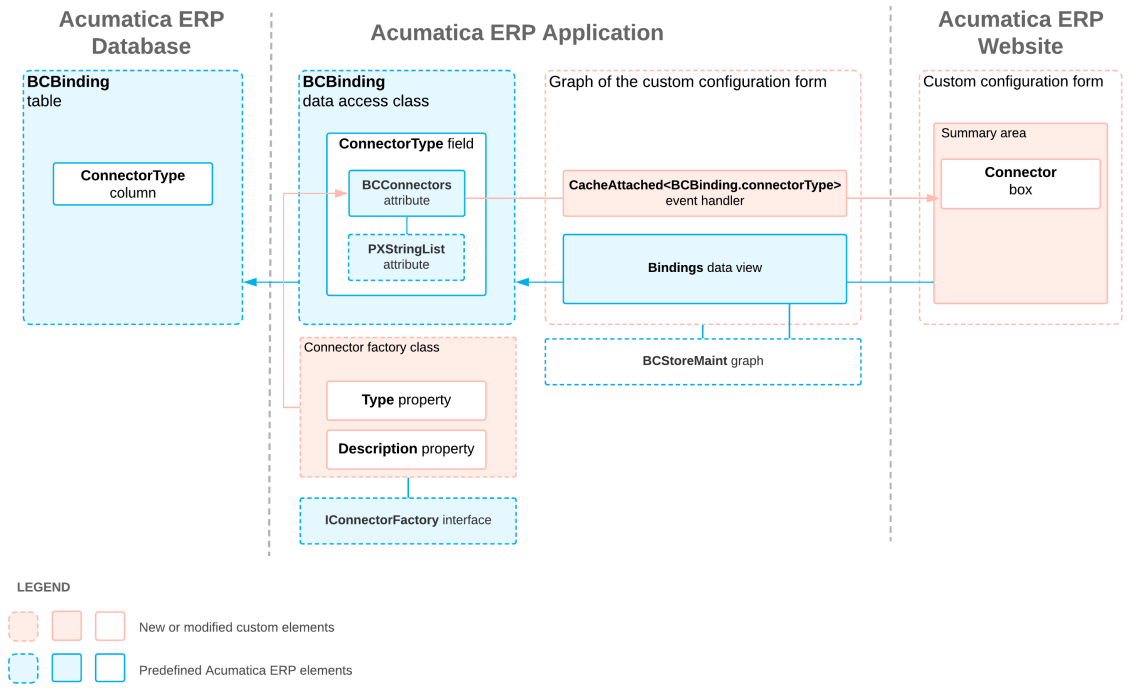


Figure: Connector box

Connector Implementation: How the Connection with the External System Is Established

A user specifies the connection settings of a store on the configuration form, such as the *BigCommerce Stores* (BC201000) form. The system saves the settings in a database table, such as the `BCBindingWooCommerce` database table, which is shown in an example in *Connector Implementation: DAC for the Connector's Configuration Settings*.

When the user clicks the **Test Connection** button on the form toolbar of the configuration form, the system retrieves the data for the connection with the store from this custom table. The connector creates the API client of the external system. The API client establishes the connection with the external system through the external API.

> The external system determines which classes you need to implement and how the connector communicates with the external system.

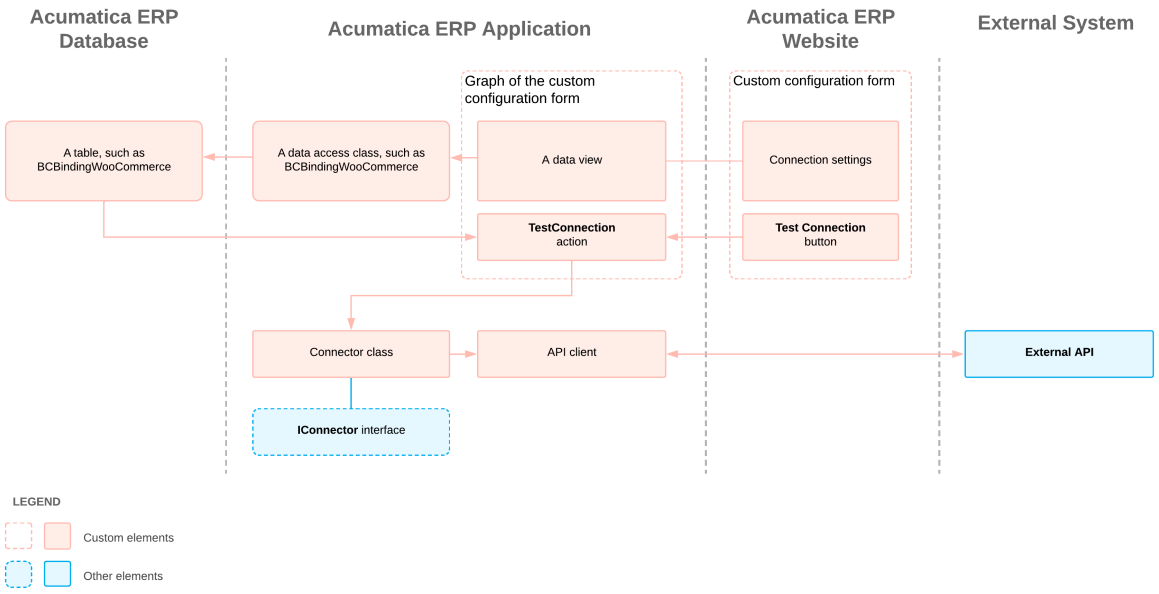


Figure: Connection with the external system

Connector Implementation: How Data Is Prepared for Synchronization

The preparation for data synchronization can be started by an automation schedule, as a result of a push notification, or manually if a user clicks **Prepare** on the toolbar of the *Prepare Data* (BC501000) form. The following sections describe the general steps involved in the preparation of data for synchronization, presented in the order in which they occur; the process diagram at the end of this topic shows the entire data preparation process.

> This topic describes the preparation for data synchronization when it is initiated on the *Prepare Data* (BC501000) form. The preparation process is slightly different if it is initiated by an automation schedule or as a result of a push notification. The preparation process initiated by an automation schedule proceeds as described in the *Fetching of the Data from Acumatica ERP and the External System* section of this topic. The preparation process that is initiated as a result of a push notification calls the `IProcessor.PullEntity()` method instead of the `IProcessor.FetchBucketsForImport()` and `IProcessor.FetchBucketsForExport()` methods.

Starting of the Data Preparation

When the data preparation is started (see Item 1 in the process diagram at the end of the topic), the system executes the `ProcessSync()` method of the `PX.Commerce.Core.BCPrepareData` graph. This method creates an instance of the connector class that corresponds to the store with which the synchronization is performed. The method also specifies the information about the current operation in a *`PX.Commerce.Core.ConnectorOperation`* object and runs the implementation of the *`IConnector.Process()`* method available in the connector class (Item 2).

For each entity that should be prepared for synchronization, the implementation of the `IConnector.Process()` method creates an instance of the processor class, initializes it with the information about the current operation, and runs its `IProcessor.Process()` method (Item 3), whose default implementation is available in the `BCProcessorSingleBase<>` class.

Fetching of the Data from Acumatica ERP and the External System

The processor calls the `FetchBucketsImport()` (see Item 4a in the process diagram) or `FetchBucketsExport()` (Item 4b) method, depending on the sync direction of the entity. These methods do the following:

- Retrieve the information about the synchronization of the entity from the *`BCEntityStats`* database table (Items 5a and 5b)
- Determine the time frame for the entities that should be prepared based on the preparation mode, which for manual preparation is specified in the **Prepare Mode** box on the *Prepare Data* (BC501000) form)
- Call the `BCProcessorSingleBase<>.FetchBucketsForImport()` (Item 6a) or `BCProcessorSingleBase<>.FetchBucketsForExport()` (Item 6b) method

The `FetchBucketsForImport()` method retrieves entity records from the external system (Item 7a). The `FetchBucketsForExport()` method obtains entity records from Acumatica ERP (Item 7b).

Saving of the Synchronization Data to the Database

For each entity record retrieved by the processor, the method creates a bucket of the entities by using the `BCProcessorBase<>.CreateBucket()` method, initializes an object that implements the `PX.Commerce.Core.IMappedEntity` interface for the entity type, and checks the preparation status of the object by using the `BCProcessorBase<>.EnsureStatus()` method.

Finally, in the `BCProcessorBase<>.UpdateEntity()` method (see Item 8 in the process diagram), the processor saves data about the synchronization status of the entity to the *`BCEntityStats`* and *`BCSyncStatus`* database tables (Item 9).

Process Diagram

The following diagram illustrates the preparation process.

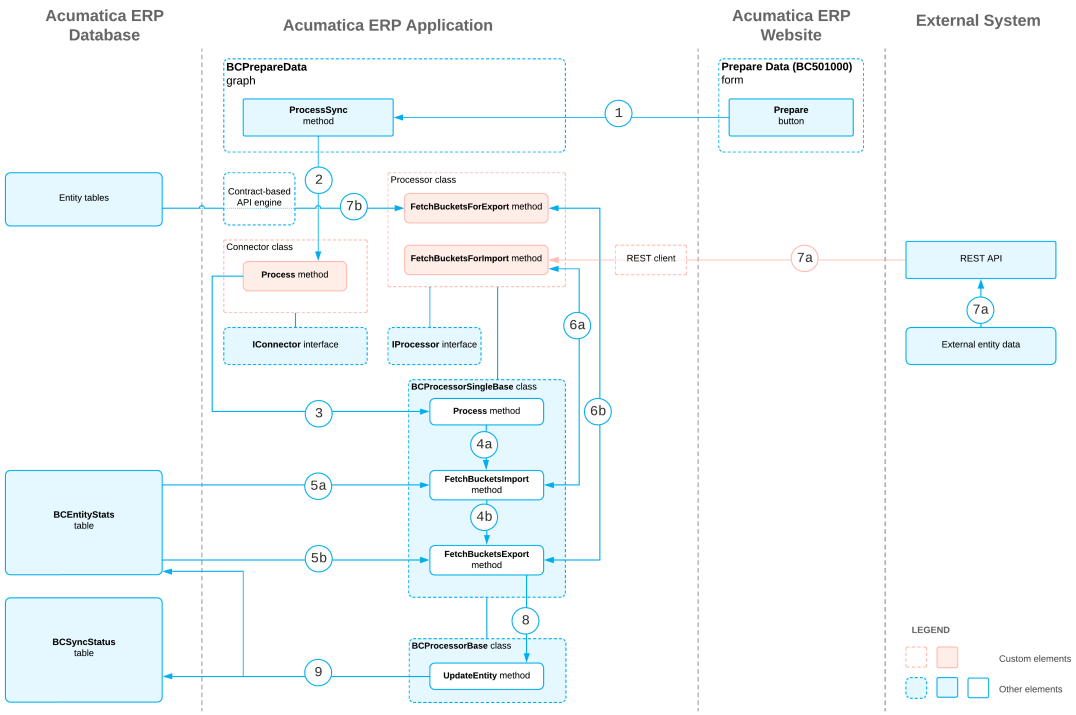


Figure: Data preparation

Connector Implementation: How the Prepared Data Is Synchronized

The synchronization of the prepared data can be started by an automation schedule, as a result of the push notification, or manually if a user clicks **Process** on the toolbar of the *Process Data* (BC501500) form. The following sections describe the general steps involved in the synchronization of the prepared data in the order in which they occur; the process diagram at the end of this topic shows the entire synchronization process.

Starting of the Synchronization

When the synchronization is started (see Item 1 in the process diagram at the end of the topic), the system executes the `ProcessSync()` method of the `PX.Commerce.Core.BCProcessData` graph. The synchronization is performed in parallel threads. The parallel processing settings are specified in the *`PXParallelProcessingOptions`* object of the `Statuses` data view of the graph.

The `ProcessSync()` method creates an instance of the connector class that corresponds to the store with which the synchronization is performed. The method also specifies the information about the current operation in a *`PX.Commerce.Core.ConnectorOperation`* object and runs the implementation of the *`IConnector.Process()`* method (Item 2) available in the connector class.

For each entity that should be synchronized, the implementation of the `IConnector.Process()` method creates an instance of the processor class, initializes it with the information about the current operation, and runs its *`IProcessor.Process()`* method (Item 3), whose default implementation is available in the `BCProcessorSingleBase<>` class.

Retrieval of the Records for the Synchronization

The processor retrieves the records for synchronization as follows:

1. Retrieves the information about synchronized records from the *BCSyncStatus* database table (see Item 4 in the process diagram).
2. By calling `BCProcessorSingleBase<>.GetBucketForExport()` (Item 5a) or `BCProcessorSingleBase<>.GetBucketForImport()` (Item 5b), pulls the records that correspond to the entity through the APIs. These methods create a bucket object for the export or import of records, respectively. The processor pulls the records from Acumatica ERP (Item 6a) by the value of the `LocalID` column of the table, and pulls the records from the external system (Item 6b) by the value of the `ExternID` column of the table.

Preprocessing of Data Before the Synchronization

If one entity has to be synchronized before the other—for example, address entities that need to be synchronized before the synchronization of customer entities—the processor synchronizes the entities that should be synchronized before the entity in the `BCProcessorBase<>.ProcessPreProcessors()` method (see Item 7 in the process diagram).

Search for the Records That Correspond to One Another

The processor searches for the records that correspond to one another in Acumatica ERP and the external system as follows:

1. If the value in `ExternID` of the *BCEntityStats* record is empty, searches for an external record that corresponds to the internal record in `BCProcessorSingleBase<>.CheckExistingForExport()` (see Item 8a in the process diagram).
2. If the value in `LocalID` of the `BCEntityStats` record is empty, searches for an internal record that corresponds to the external record in `BCProcessorSingleBase<>.CheckExistingForImport()` (Item 8b).

> The connector performs the following actions during the search:
>
> a. Pulls records from the destination system by using a unique key, such as the customer's email address, the product name, or the order's external reference number (Item 9a or 9b).
>
> b. If no records are found, continues with the synchronization of the new record, as described in the following section.
>
> c. If multiple records are returned from the destination system, throws an error and aborts the synchronization.
>
> d. If only one record is returned from the destination system, checks whether this record has already been mapped to any other record by using the *BCSyncStatus* table (Item 10a or 10b) as follows:
>
> a. If there is a record that has already been mapped to another record, the processor throws an error and aborts the synchronization.
>
> b. If there are no mapped records, the processor maps the currently processed record to the record it found. The record will be synchronized as an existing record, as described below.

Control of the Synchronization Direction

The processor compares the time stamps of the synchronized records and decides in which direction these records should be synchronized as follows:

1. If both the external records and the internal records have been changed (which is indicated by their time stamps), the synchronization from the primary system to the secondary system is performed.

2. If neither an external record nor an internal record has been changed (which is indicated by their time stamps), the synchronization from the primary system to the secondary system is performed only if forced synchronization is being performed.

   > Forced synchronization is performed if a user clicks **Sync** on the toolbar of the *Sync History* (BC301000) form.

3. If only one time stamp has been changed, the processor synchronizes from the system of the changed record to the system of the record that has not been changed.

Then an implementation of the `BCProcessorSingleBase<>.ControlDirection()` method (see Item 11 in the process diagram) is called, in which you can add additional logic to the control of the synchronization direction.

When a direction is chosen, in the `BCProcessorBase<>.EnsureSync()` method (Item 12), the processor updates the `SyncInProcess` flag of the *`BCSyncStatus`* table (Item 13). This locks the record so that it cannot be synchronized from the other threads. If the record has been locked already by the other process, the connector aborts the operation and proceeds to the processing of the next record.

Import of Records

The processor imports the data to Acumatica ERP in the `BCProcessorSingleBase<>.ProcessImport()` method (see Item 14a in the process diagram), which internally performs the following steps:

1. For the records that have been deleted, removes the related records in Acumatica ERP in the `BCProcessorBase<>.DeleteRelatedEntities()` method.
2. Maps the properties of the external object to the properties of the Acumatica ERP object in the `BCProcessorBase<>.MapBucketImport()` method.
3. Applies the mappings that have been defined by the user on the *Entities* (BC202000) form in the `BCProcessorBase<>.RemapBucketImport()` method.
4. In the `BCProcessorBase<>.ValidateImport()` method, validates the entity that is going to be saved to the Acumatica ERP database to ensure that no required fields are left empty.
5. Saves the data to the database in `BCProcessorSingleBase<>.SaveBucketImport()` (Item 15a).

   > At the end of the implementation of the `SaveBucketImport()` method, you need to call the `BCProcessorBase<>.UpdateStatus()` method to update the time stamps and IDs of the synchronized records. For details, see *Update of the Synchronization Status*.

Export of Records

The processor exports the data from Acumatica ERP to the external system in the `BCProcessorSingleBase<>.ProcessExport()` method (see Item 14b in the process diagram), which internally performs the following steps:

1. For the records that have been deleted, removes the related records in the external system in the `BCProcessorBase<>.DeleteRelatedEntities()` method.
2. Maps the properties of the Acumatica ERP object to the properties of the external object in `BCProcessorBase<>.MapBucketExport()` method.
3. Applies the mappings that have been defined by the user on the *Entities* (BC202000) form in the `BCProcessorBase<>.RemapBucketExport()` method.
4. In the `BCProcessorBase<>.ValidateExport()` method, validates the record that is going to be saved to the external system to ensure that no required fields are left empty.

5. Saves the data to the external system in `BCProcessorSingleBase<>.SaveBucketExport()` (Item 15b).

> At the end of the implementation of the `SaveBucketExport()` method, you need to call the `BCProcessorBase<>.UpdateStatus()` method to update the time stamps and IDs of the synchronized records. For details, see the next section.

Update of the Synchronization Status

In the `BCProcessorBase<>.UpdateStatus()` method (see Item 16 in the process diagram), the processor updates the *`BCSyncStatus`* record (Item 17) with the result of the operation as follows:

- If the synchronization was successful, the processor updates the record to reflect the new time stamps and IDs. `PendingSync` is set to `false`.
- If the synchronization has failed, the processor updates the status to *Failed*, updates the error message, and increments the number of failed attempts.
- If the synchronization of this record has failed more than the configured maximum number of attempts, the processor automatically assigns the *Skipped* status to the record.

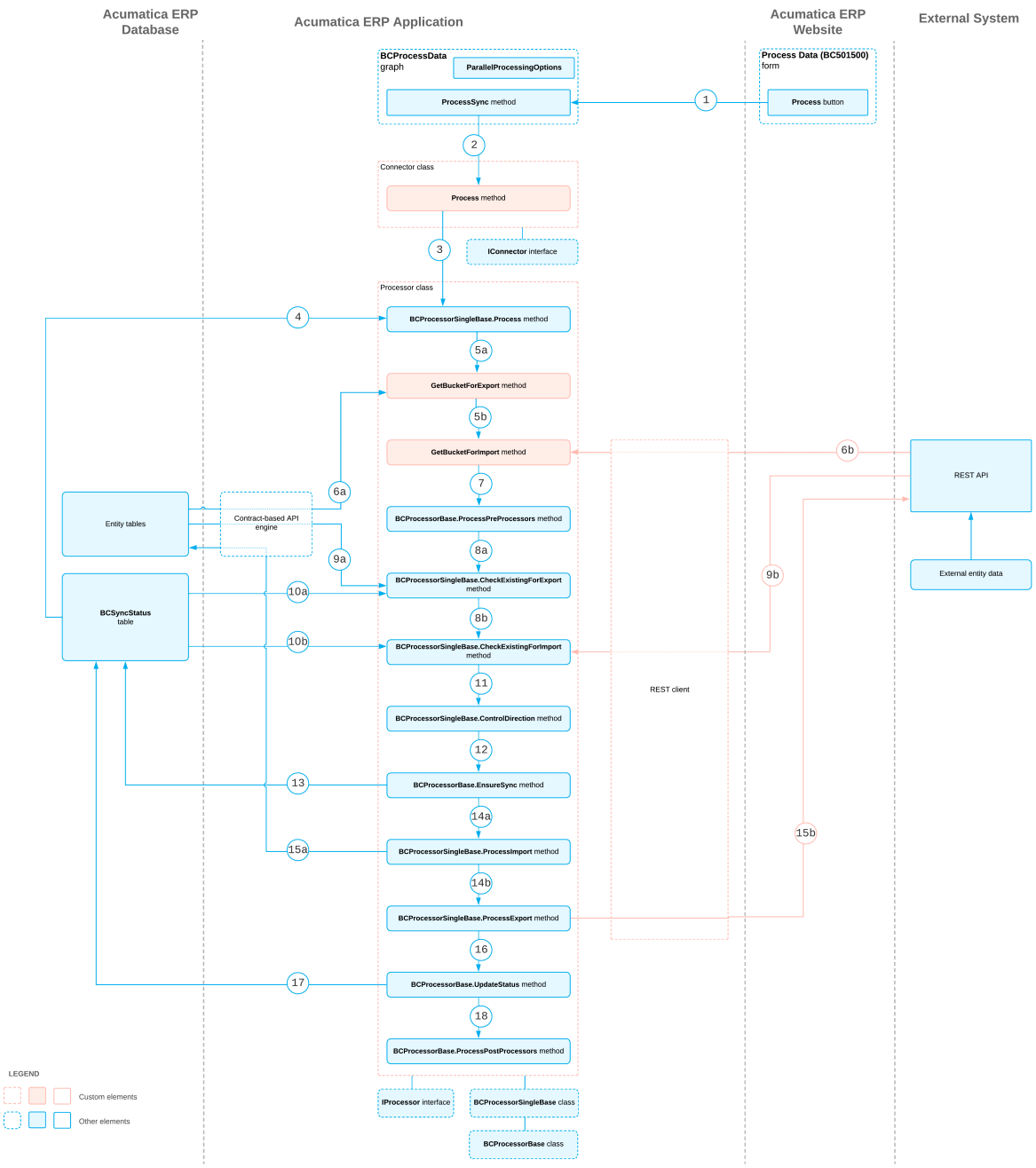
The processor unlocks the record by updating the `SyncInProcess` flag of the `BCSyncStatus` table. The processor then saves all the changes to the database.

Synchronization of the Dependent Records

The processor synchronizes the entities that should be synchronized after the entity in the `BCProcessorBase<>.ProcessPostProcessors()` method (see Item 18 in the process diagram). If post-processing has failed, the synchronization of the current entity is not affected.

Process Diagram

The following diagram illustrates the process of data synchronization.


Acumatica ERP Database
Acumatica ERP Application
Acumatica ERP Website
External System
BCProcessData graph
ParallelProcessingOptions
ProcessSync method
Process Data (BC501500) form
Process button
Connector class
Process method
IConnector interface
Processor class
BCProcessorSingleBase.Process method
GetBucketForExport method
GetBucketForImport method
BCProcessorBase.ProcessPreProcessors method
BCProcessorSingleBase.CheckExistingForExport method
BCProcessorSingleBase.CheckExistingForImport method
BCProcessorSingleBase.ControlDirection method
BCProcessorBase.EnsureSync method
BCProcessorSingleBase.ProcessImport method
BCProcessorSingleBase.ProcessExport method
BCProcessorBase.UpdateStatus method
BCProcessorBase.ProcessPostProcessors method
IProcessor interface
BCProcessorSingleBase class
BCProcessorBase class
Entity tables
Contract-based API engine
BCSyncStatus table
REST client
REST API
External entity data
LEGEND
Custom elements
Other elements

Implementing Real-Time Synchronization for a Connector

With real-time synchronization between Acumatica ERP and an external system, Acumatica ERP attempts to prepare data or to prepare and process data as soon as a change occurs in either system. Depending on the entity involved and your company's processes, real-time synchronization can involve import or export, or it can be bidirectional. Real-time import relies on webhooks that Acumatica ERP receives from an external system, and real-time export makes use of the push notification mechanism available in Acumatica ERP.

Real-Time Synchronization for a Connector: General Information

Real-time synchronization between Acumatica ERP and an external system ensures that any updates made in one system are reflected in the other system without significant delay, keeping the data consistent and current across these systems. You implement real-time import through webhooks. You implement real-time import through webhooks, and real-time export by using push notifications.

Learning Objectives

In this chapter, you will learn how to implement real-time synchronization for a connector between Acumatica ERP and an external system.

Applicable Scenarios

You implement real-time synchronization to ensure that all needed data is exported and imported to the appropriate system in a timely manner.

Implementation of Webhooks in the Connector

In a connector, you can implement real-time import for particular entities through webhooks. For these entities, you need to adjust the connector class and the processor classes that correspond to these entities by performing the following general steps:

1. You make sure that the connector class implements the following methods: `StartWebhook()`, `ProcessHook()`, and `StopWebhook()`.
2. For each processor class that needs to support real-time import through webhooks, you make sure the following requirements are met:
   - The `BCProcessor` attribute of the processor has the `Direction` property set to *SyncDirection.Bidirect* or *SyncDirection.Import*.
   - The `FetchBucketsForImport()`, `GetBucketForImport()`, `MapBucketImport()`, `SaveBucketImport()`, and `PullEntity()` methods of the processor have been implemented.
3. For each processor class that needs to support real-time import through webhooks, you modify the `BCProcessorRealtime` attribute of the processor so that the `HookSupported` property is set to `true`. In the `WebHookType` property, you specify the type of the object to be used for the deserialization of the JSON object received in the webhook. In the `WebHooks` property, specify the list of webhook scopes.

   The following code shows an example of the attribute for the customer processor class.

```
[BCProcessorRealtime(
```

```
    HookSupported = true,
    WebHookType = typeof(CustomerData),
    WebHooks = new String[]
    {
```

```
            "customer.created",
            "customer.updated",
            "customer.deleted"
        }
    )]
    public class WooCustomerProcessor :
        BCProcessorSingleBase<WooCustomerProcessor,
            WooCustomerEntityBucket, MappedCustomer>, IProcessor
    {
    }
```

You can see this code on *GitHub*.

Implementation of Push Notifications for a Connector

To implement push notifications in Acumatica ERP to support real-time export in a connector, you need to perform the following general steps:

1. **Defining the data for real-time synchronization.**

   You need to create generic inquiries or select existing ones that will define the data whose changes trigger the sending of notifications from Acumatica ERP to the connector.

   On the *Push Notifications* (SM302000) form, you can select *Commerce* in the **Destination Name** box of the Summary area; on the **Generic Inquiries** tab, you review the list of generic inquiries that are predefined for the commerce push notifications. If the list of generic inquiries is sufficient for your connector, we recommend that you use the predefined *Commerce* notification destination.

   If you need a different list of generic inquiries for the entities that you want the system to send push notifications for, you create a custom push notification definition, as described in *Push Notifications: General Information*.

   You need to include the created generic inquiries and push notification definitions in the customization project of your extension library. For details about adding these items to the customization project, see *Generic Inquiries in a Customization Project: General Information* and *Push Notifications: Inclusion in a Customization Project*.

2. **Supporting push notifications in the connector.**

   For the entities for which you need to support real-time export through push notifications, you need to adjust the corresponding processor classes so that they support real-time synchronization. For details about processor classes, see *Connector Implementation: Processor Classes*.

   For each processor class that needs to support real-time export through push notifications, you need to make sure that the processor class supports the export of records from Acumatica ERP by checking the following requirements:

   - The `BCProcessor` attribute of the processor has the `Direction` property set to *SyncDirection.Bidirect* or *SyncDirection.Export*.
   - The `FetchBucketsForExport()`, `GetBucketForExport()`, `MapBucketExport()`, `SaveBucketExport()`, and `PullEntity()` methods of the processor have been implemented. The following code shows an example of the implementation of the `PullEntity()` method.

```
public override async Task<MappedCustomer> PullEntity(Guid? localID,
    Dictionary<string, object> externalInfo,
        CancellationToken cancellationToken = default)
{
    PX.Commerce.Core.API.Customer impl =
        cbapi.GetByID<PX.Commerce.Core.API.Customer>(localID);
    if (impl == null) return null;
```

```
            MappedCustomer obj = new MappedCustomer(impl, impl.SyncID,
                impl.SyncTime);

            return obj;
        }
```

For each processor class that needs to support real-time synchronization through push notifications, you modify the `BCProcessorRealtime` attribute of the processor to set the `PushSupported` property to *true*. In the `PushSources` property, you specify the names of the generic inquiries to be used for push notifications. In the `PushDestination` property, you specify the name of the push notification destination, which is *Commerce* if you use the predefined notification destination.

The following code shows an example of the attribute for the customer processor class.

```
[BCProcessorRealtime(
    PushSupported = true,
    PushSources = new String[] { "BC-PUSH-Customers" },
    PushDestination = WCCaptions.PushDestination,
```

```
)]
public class WooCustomerProcessor :
    BCProcessorSingleBase<WooCustomerProcessor,
        WooCustomerEntityBucket, MappedCustomer>, IProcessor
{
}
```

You can see the code from this section on *GitHub*.

Real-Time Synchronization for a Connector: How Webhooks and Push Notifications Are Used

In this topic, you can find information about the process of real-time synchronization through a connector between Acumatica ERP and an external system.

Processing of Commerce Webhooks

When an administrator starts real-time synchronization on the *Entities* (BC202000) form by clicking **Start Real-Time Sync** on the form toolbar, the system calls the `IConnector.StartWebhook` method. The system uses this method to initialize the webhook scopes in the external system. Each webhook scope is a situation when the webhook, such as a customer update, is sent. The webhook scopes are specified in the `Webhooks` parameter of the `BCProcessorRealtime` attribute of the processor.

When an external system sends a webhook request, Acumatica ERP uses the webhook handler class to receive the webhook request and to place the message in a queue in the queue broker (which is RabbitMQ by default). The queue has the *commerceQueue* prefix. You can monitor the status of this queue on the *System Queue Monitor* (SM302010) form. The messages are then processed as described in the *Real-Time Synchronization* section of this topic.

For BigCommerce, Shopify, or other compatible systems, the webhook handler class is the `BCWebHookHandler` internal class. The class expects the following data in the query:

- *type*: The connector type
- *company*: The company ID in Acumatica ERP

- *validation*: The hash code that is generated by Acumatica ERP, which indicates that the message is expected

For BigCommerce webhooks, the class expects this data in headers. For Shopify webhooks, the class expects this data in the URL parameters.

If you cannot configure the external system to send webhook requests in one of these formats, you can implement a custom webhook handler class.

When the administrator stops real-time synchronization on the *Entities* form by clicking **Stop Real-Time Sync** on the form toolbar, the system calls the `IConnector.StopWebhook` method.

Processing of Commerce Push Notifications

For real-time export, Acumatica ERP includes a number of predefined generic inquiries that define the data changes for which Acumatica ERP should send notifications to the Acumatica Commerce Framework. These generic inquiries are listed on the **Generic Inquiries** tab of the *Push Notifications* (SM302000) form for the *Commerce* notification destination, which is predefined in the system. The names of these generic inquiries begin with the *BC-PUSH* prefix.

You can create custom generic inquiries for the monitoring of entities that are not included in the predefined inquiries. You can then create a custom notification destination of the *Commerce Push Destination* type on the *Push Notifications* form. In the list of inquiries for this notification destination, you include both the custom generic inquires and the needed predefined generic inquiries.

For a data query that is defined by a predefined or custom generic inquiry, when the system identifies a change in the results, the system generates a notification and adds it to a queue in the queue broker, which is RabbitMQ by default. This queue has the *primaryQueue* prefix and is used for all push notifications generated by Acumatica ERP.

An Acumatica ERP background thread processes the notifications from this queue and sends the commerce notifications to the notification destinations of the *Commerce Push Destination* type. By default, the system includes only one notification destination of the *Commerce Push Destination* type, which is the *Commerce* notification destination. The *Commerce Push Destination* type is defined in the `PX.Commerce.Core.Model.Notification.BCPushNotificationDestination` class, which implements the *`PX.PushNotifications.NotificationSenders.IPushNotificationSender`* interface.

The notification destination of the *Commerce Push Destination* type processes the commerce notifications and places them in another queue in the queue broker (which is RabbitMQ by default). The queue has the *commerceQueue* prefix, and you can monitor its status on the *System Queue Monitor* (SM302010) form. The messages are then processed as described in the next section.

Real-Time Synchronization

The commerce background thread processes the commerce notifications from the queue with the *commerceQueue* prefix. To improve the performance of the commerce connector, this thread works with a 20-second delay. That is, when the thread receives a message, it waits for 20 seconds before it starts to process this message. For example, if a user has saved a customer record three times in the last 15 seconds, the system generates three push notifications about the changes. Because the second and third modification all happened within the 20-second delay of the first push notification, the commerce connector synchronizes only the version of the customer record from the most recent push notification.

The commerce background thread updates the `BCSyncStatus` database table with the information about the modified record. You can view the data from this table on the *Sync History* (BC301000) form. Depending on the option selected in the **Real-Time Mode** box on the *Entities* (BC202000) form, the commerce background thread does one of the following:

- If *Prepare* is selected, places the record in the *`BCSyncStatus`* table with the *Pending* status. For details about the preparation of data for synchronization, see *Connector Implementation: How Data Is Prepared for Synchronization*.

- If *Prepare & Process* is selected, after placing the record in the `BCSyncStatus` table, immediately starts the synchronization of the record. For more information about data synchronization, see *Connector Implementation: How the Prepared Data Is Synchronized*.

The processing of import and export notifications is implemented with the *`IConnector.ProcessHook()`* and *`IConnector.ProcessPush()`* methods, respectively; the methods' default implementations are available in `PX.Commerce.Core.BCConnectorBase<TConnector>`. These default implementations call the *`IProcessor.ProcessHook()`* and *`IProcessor.ProcessPush()`* methods, respectively, which are implemented in `PX.Commerce.Core.BCProcessorBase<>`.

If any error occurs during the preparation or processing of the data for synchronization, the error is displayed on the **System Events** tab of the *System Monitor* (SM201530) form.